人工智能与深度学习

用于自然语言处理的深度学习技术

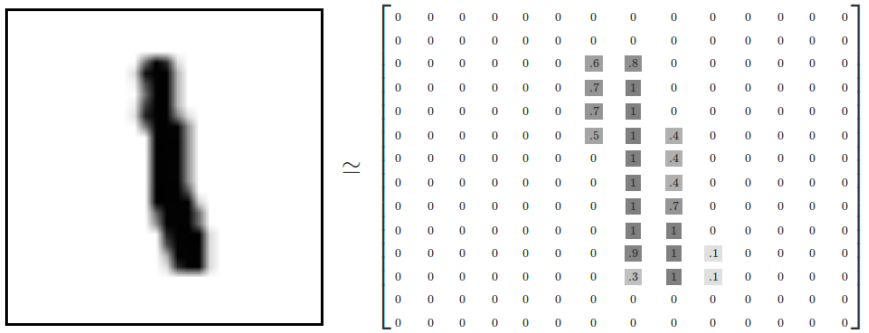
华中科技大学 机械科学与工程学院
陈伟

2026

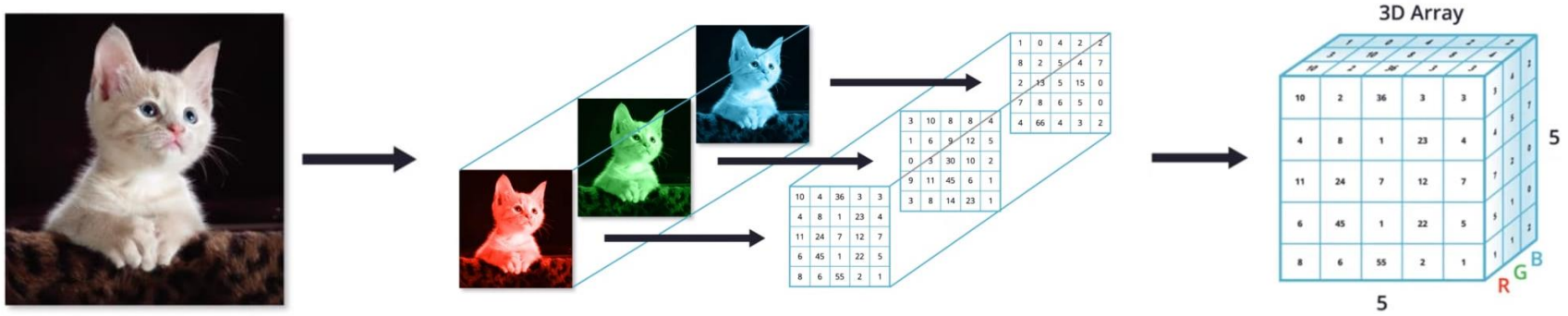
词嵌入
(Word Embedding)

回顾：图像的表示

- **灰度图像可以直接用二维数组表示**
  - 回顾： MNIST 中的手写数字可用 28x28 的数组表示



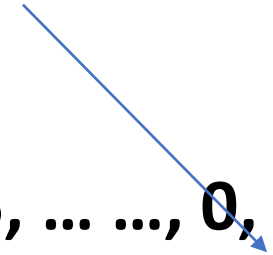
- **彩色图像可用三维数组表示**
  - RGB三种颜色（三个通道）分别可用“灰度图像”表示



文字如何用数组表示?

- The quick brown fox jumps over the lazy dog.

- fox, jump, red, rule, conduct, philosophy, Newton, HUST, …



[0, 0.13, 0.25, … …, 0, 0.04]

文字如何用数组表示?

- Word embedding：把单词转换为对应的向量

- 直观的办法：one-hot encoding

Vocabulary		one-hot encodings
dog	1	[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
cat	2	[0, 1, 0, 0, 0, 0, 0, 0, 0, 0]
person	3	[0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
holding	4	[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]
tree	5	[0, 0, 0, 0, 1, 0, 0, 0, 0, 0]
computer	6	[0, 0, 0, 0, 0, 1, 0, 0, 0, 0]
using	7	[0, 0, 0, 0, 0, 0, 1, 0, 0, 0]

词与词之间的相似性和关系

- **One-hot encoding 不容易反映词与词之间的相似性，及其它复杂的关系**

- **相似的词在句子中的位置及作用往往类似**
  - 猫 & 狗；红 & 绿；走 & 跑
  - 我们希望相似的词对应的向量也比较接近

词与词之间的相似性和关系

- **词与词之间的关系**
  - 北京 - 中国 = 巴黎 - 法国
  - King – Man + Woman = Queen

- **词与词之间的关系**
  - 吃/蒸 + 包子/饺子
  - 白色的/联名款的 + 手机壳/充电宝

语言统计概率分布问题

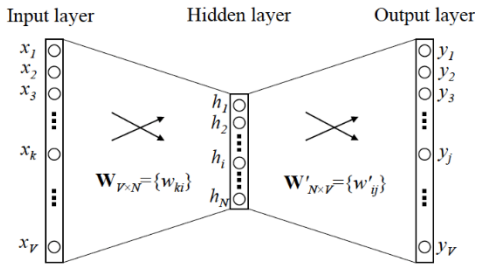
- **如何发现词与词之间的相似性和关系**
  - 从大量的文本段落中学习，学习词语序列的统计规律

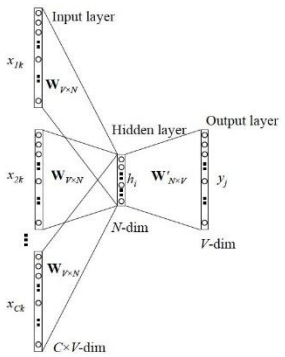
- **两个关键的语言模型**
  - **给定一个词，预测其上下文的概率分布**
    - 也被称为 Continuous Bag-of-Word Model，简称 CBOW
  - **给定上下文，预测这个词的概率分布**
    - 也被称为 Continuous Skip-gram Model，简称 Skip-gram

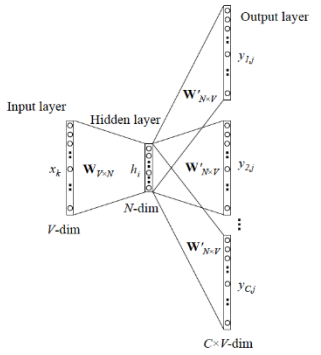
word2vec

- **word2vec 是Word Embedding 的经典方法之一**

- **基本思路**
  - 先使用一套 one-hot encoder 的词语向量表示
  - 训练一个类似神经网络的模型，预测词语上下文的概率分布（CBOW），也通过上下问预测该词语的概率分布（Skip-gram）
  - 该模型的一些**参数**，可以作为该词语的向量表示







Word2vec 及其它模型

- **关于 word2vec 模型，详见 [1411.2738] word2vec Parameter Learning Explained (arxiv.org)**

- **仅靠 word2vec 模型解决不了的问题**
  - 多义词
  - 中文语言的词语划分

- **后来又发展出了 GloVe 等新的词语向量化方式**

词嵌入

- 英文词语一般用 100 – 300 维的向量来表示

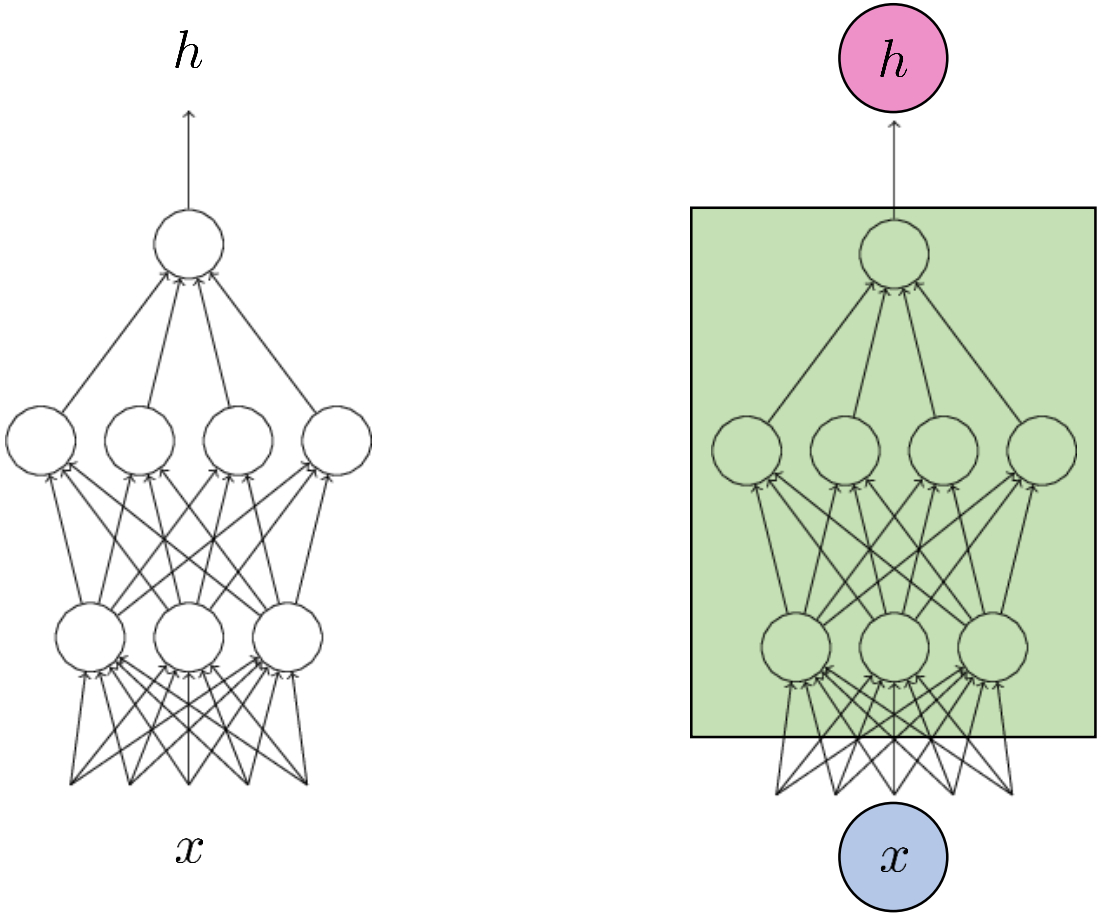
	0	1	2	3	4	5	6	7	8	9	...	290	291	292	
fox	-0.348680	-0.077720	0.177750	-0.094953	-0.452890	0.237790	0.209440	0.037886	0.035064	0.899010	...	-0.283050	0.270240	-0.654800	0.10
ham	-0.773320	-0.282540	0.580760	0.841480	0.258540	0.585210	-0.021890	-0.463680	0.139070	0.658720	...	0.464470	0.481400	-0.829200	0.35
brown	-0.374120	-0.076264	0.109260	0.186620	0.029943	0.182700	-0.631980	0.133060	-0.128980	0.603430	...	-0.015404	0.392890	-0.034826	-0.72
beautiful	0.171200	0.534390	-0.348540	-0.097234	0.101800	-0.170860	0.295650	-0.041816	-0.516550	2.117200	...	-0.285540	0.104670	0.126310	0.12
jumps	-0.334840	0.215990	-0.350440	-0.260020	0.411070	0.154010	-0.386110	0.206380	0.386700	1.460500	...	-0.107030	-0.279480	-0.186200	-0.54
eggs	-0.417810	-0.035192	-0.126150	-0.215930	-0.669740	0.513250	-0.797090	-0.068611	0.634660	1.256300	...	-0.232860	-0.139740	-0.681080	-0.37
beans	-0.423290	-0.264500	0.200870	0.082187	0.066944	1.027600	-0.989140	-0.259950	0.145960	0.766450	...	0.048760	0.351680	-0.786260	-0.36
sky	0.312550	-0.303080	0.019587	-0.354940	0.100180	-0.141530	-0.514270	0.886110	-0.530540	1.556600	...	-0.667050	0.279110	0.500970	-0.27
bacon	-0.430730	-0.016025	0.484620	0.101390	-0.299200	0.761820	-0.353130	-0.325290	0.156730	0.873210	...	0.304240	0.413440	-0.540730	-0.03
breakfast	0.073378	0.227670	0.208420	-0.456790	-0.078219	0.601960	-0.024494	-0.467980	0.054627	2.283700	...	0.647710	0.373820	0.019931	-0.03
toast	0.130740	-0.193730	0.253270	0.090102	-0.272580	-0.030571	0.096945	-0.115060	0.484000	0.848380	...	0.142080	0.481910	0.045167	0.05
today	-0.156570	0.594890	-0.031445	-0.077586	0.278630	-0.509210	-0.066350	-0.081890	-0.047986	2.803600	...	-0.326580	-0.413380	0.367910	-0.26
blue	0.129450	0.036518	0.032298	-0.060034	0.399840	-0.103020	-0.507880	0.076630	-0.422920	0.815730	...	-0.501280	0.169010	0.548250	-0.31
green	-0.072368	0.233200	0.137260	-0.156630	0.248440	0.349870	-0.241700	-0.091426	-0.530150	1.341300	...	-0.405170	0.243570	0.437300	-0.46
kings	0.259230	-0.854690	0.360010	-0.642000	0.568530	-0.321420	0.173250	0.133030	-0.089720	1.528600	...	-0.470090	0.063743	-0.545210	-0.19
dog	-0.057120	0.052685	0.003026	-0.048517	0.007043	0.041856	-0.024704	-0.039783	0.009614	0.308416	...	0.003257	-0.036864	-0.043878	0.00
sausages	-0.174290	-0.064869	-0.046976	0.287420	-0.128150	0.647630	0.056315	-0.240440	-0.025094	0.502220	...	0.302240	0.195470	-0.653980	-0.29
lazy	-0.353320	-0.299710	-0.176230	-0.321940	-0.385640	0.586110	0.411160	-0.418680	0.073093	1.486500	...	0.402310	-0.038554	-0.288670	-0.24
love	0.139490	0.534530	-0.252470	-0.125650	0.048748	0.152440	0.199060	-0.065970	0.128830	2.055900	...	-0.124380	0.178440	-0.099469	0.00
quick	-0.445630	0.191510	-0.249210	0.465900	0.161950	0.212780	-0.046480	0.021170	0.417660	1.686900	...	-0.329460	0.421860	-0.039543	0.15

20 rows × 300 columns

递归神经网络
(Recurrent Neural Network，RNN)

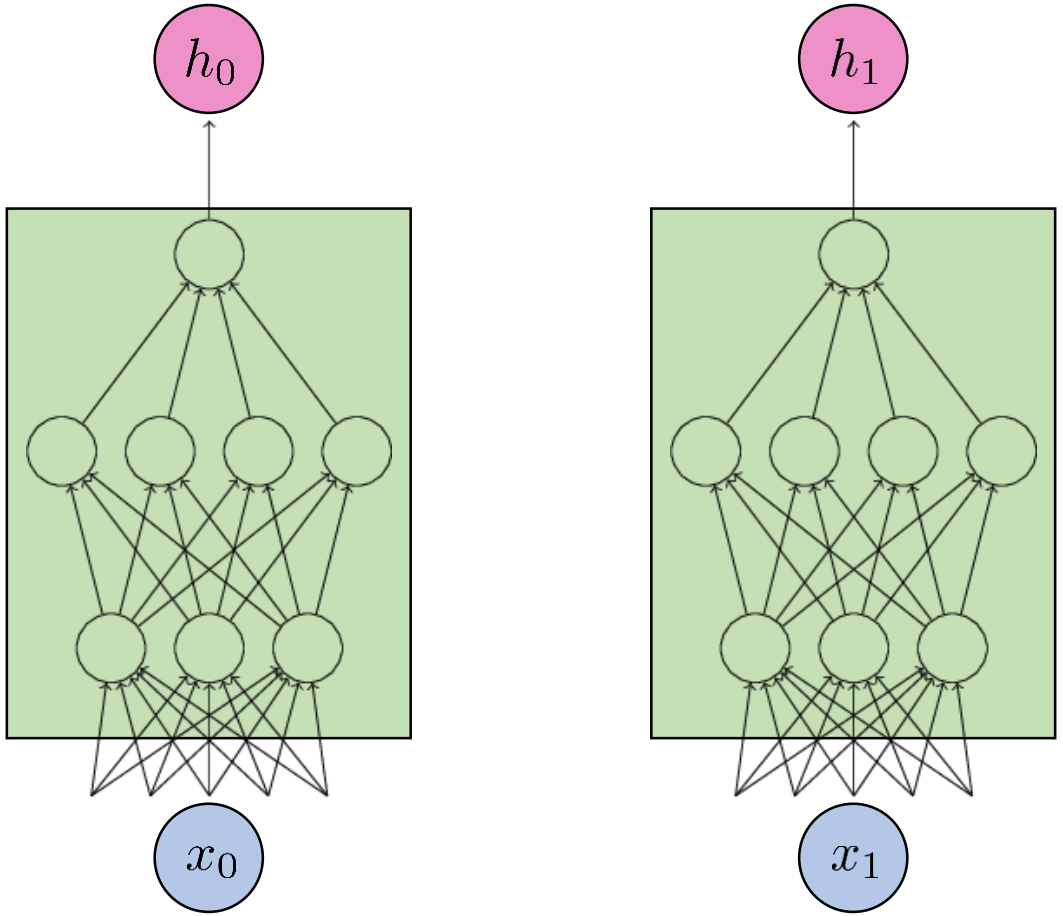
神经网络

- 此前我们讲到的神经网络模型，输入是一个向量（或张量），输出是一个向量（或张量）



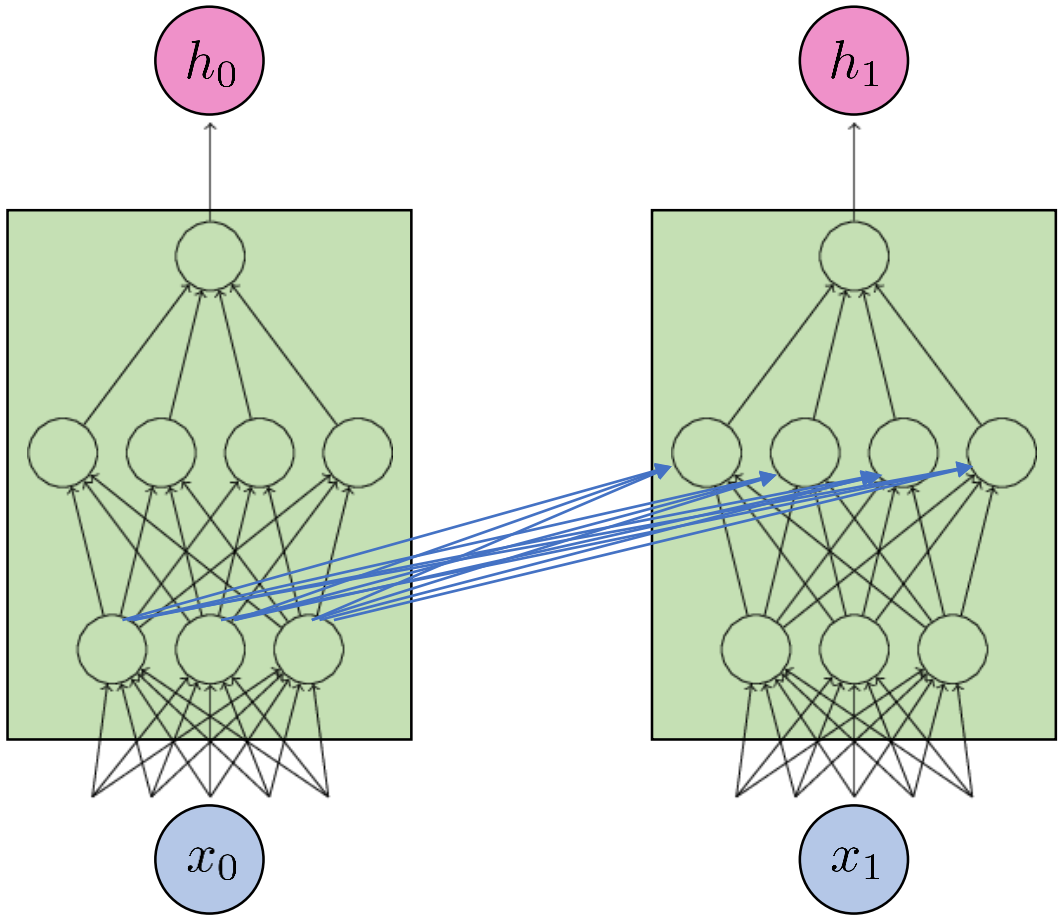
神经网络

- 在语言等模型中，我们希望输入是向量序列，输出也是向量序列

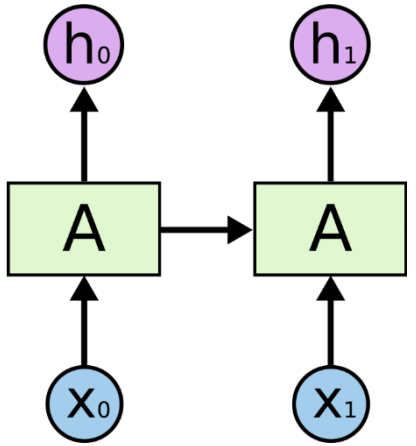


RNN

- **在神经网络中的某隐藏层，让序列信息传递下去**

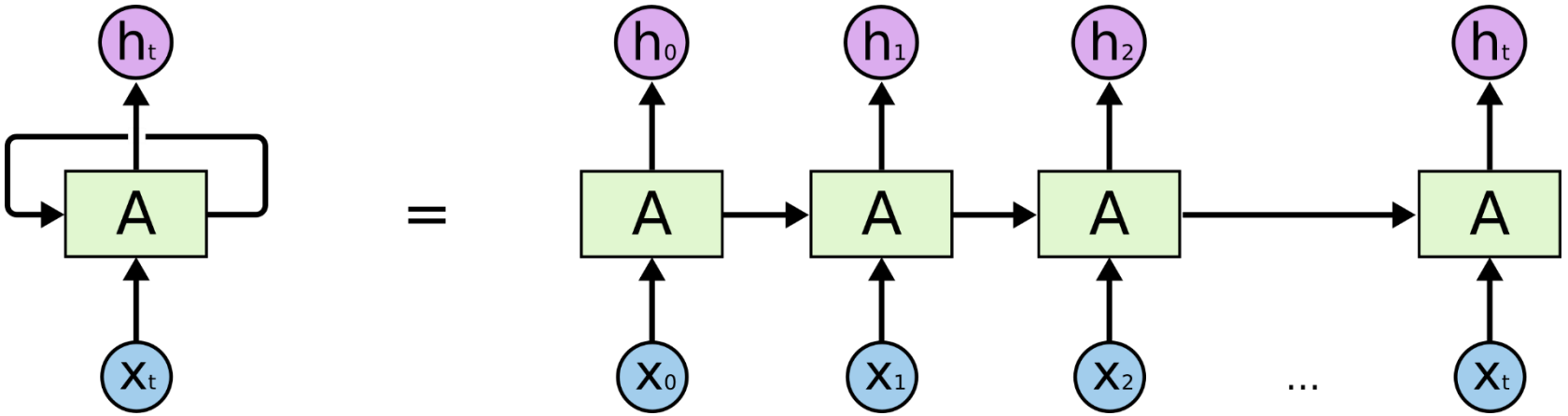


简化的画法



RNN

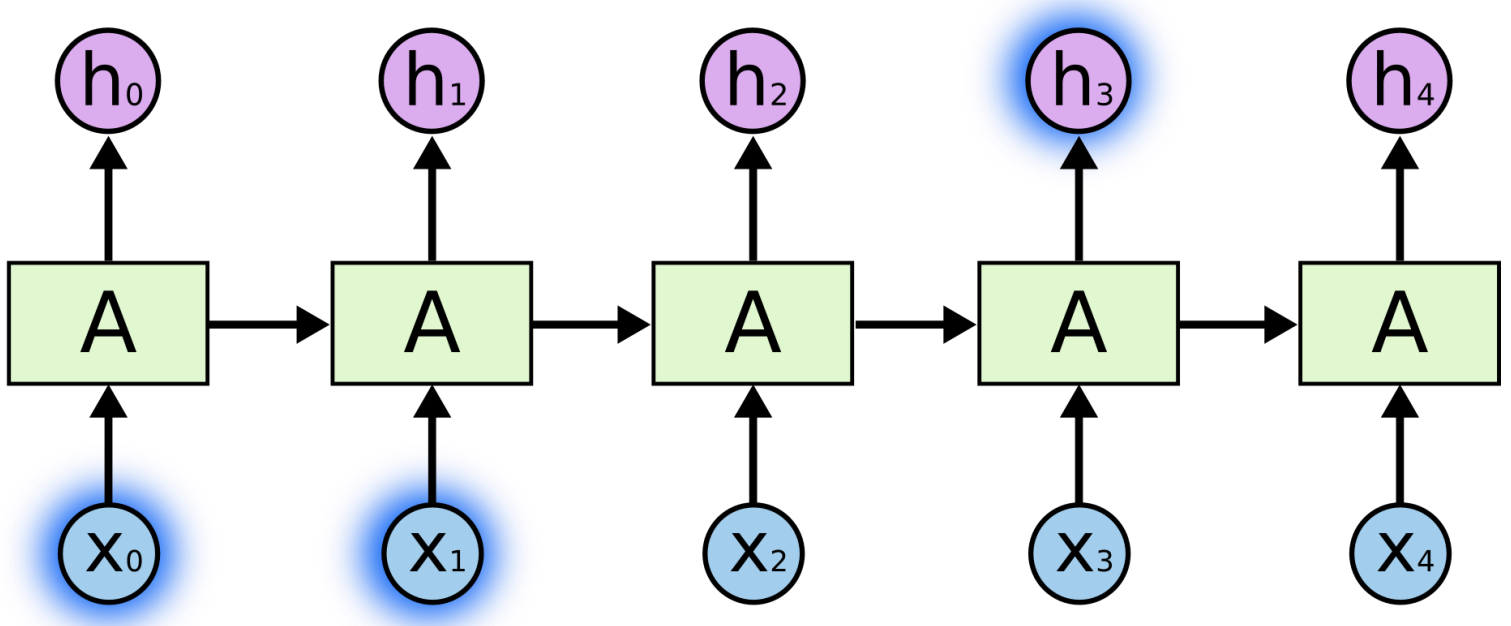
- **“序列-序列”的递归神经网络 RNN**



进一步
简化的画法

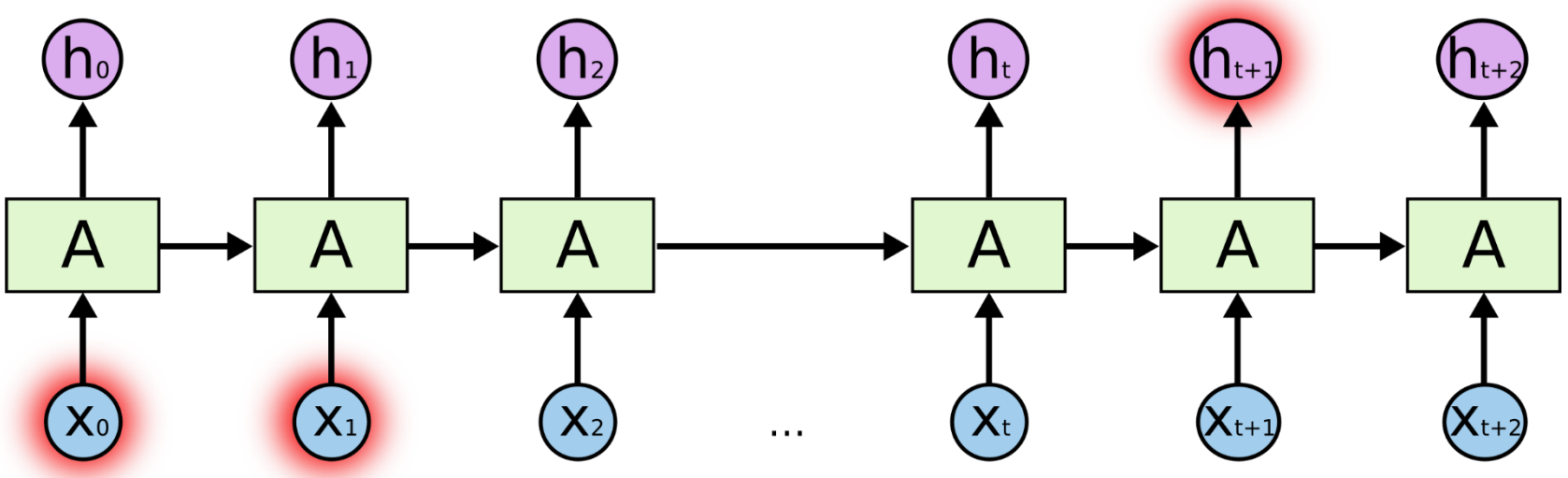
RNN

- **RNN 可以将序列中过去的信息与当前的信息联系起来**
  - 考虑以下填空：The clouds are in the sky.



RNN

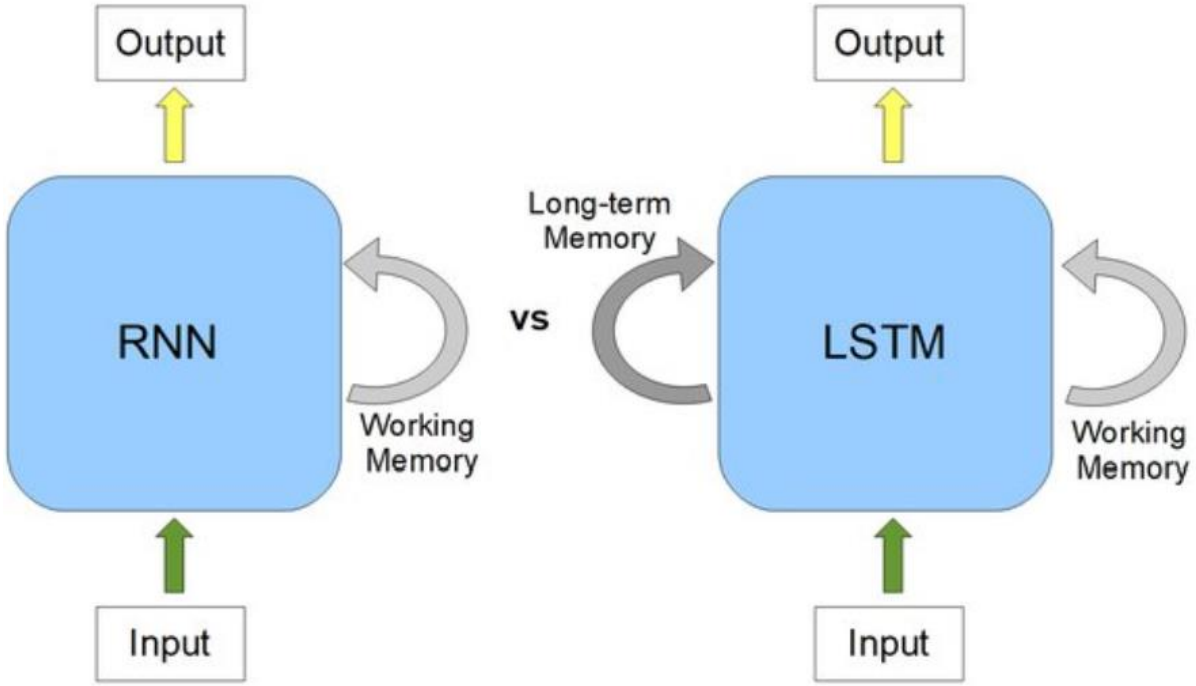
- **当需要用到的信息在序列中的位置过于久远时，RNN 的效果并不太好**
  - 考虑以下填空：I grew up in France… I speak fluent French.



长短期记忆
(Long Short-Term Memory，LSTM)
神经网络

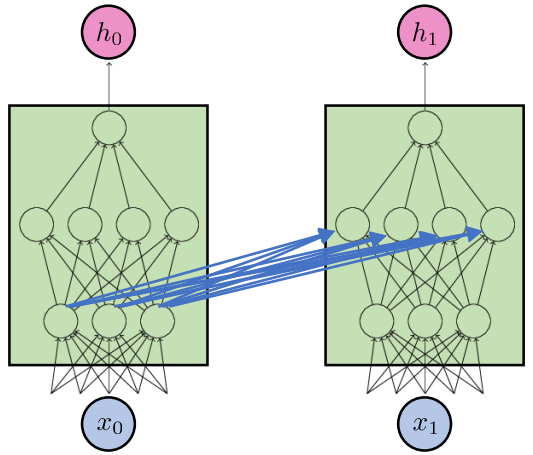
LSTM

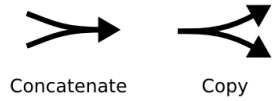
- **LSTM 指的是一类具有“长期记忆”的 RNN**



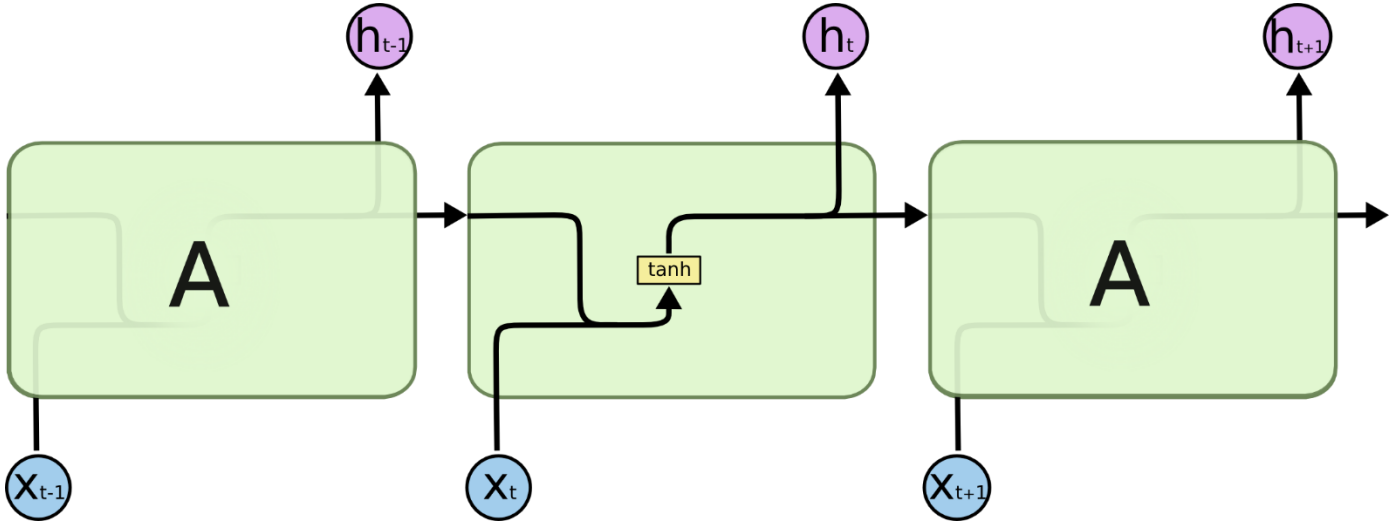
RNN

- **一般的 RNN 可以用下图表示**





Concatenate：向量拼接



回顾：激活函数

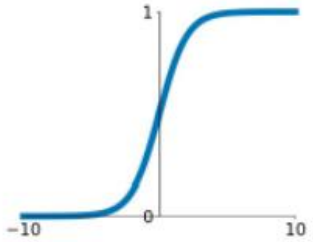
-

双曲正切函数

$$\tanh\ x = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

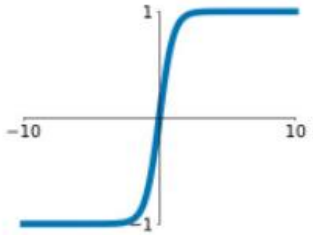
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



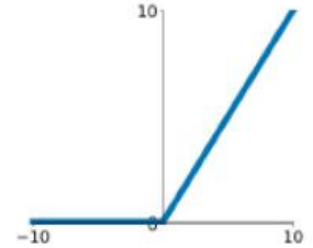
tanh

$$\tanh(x)$$



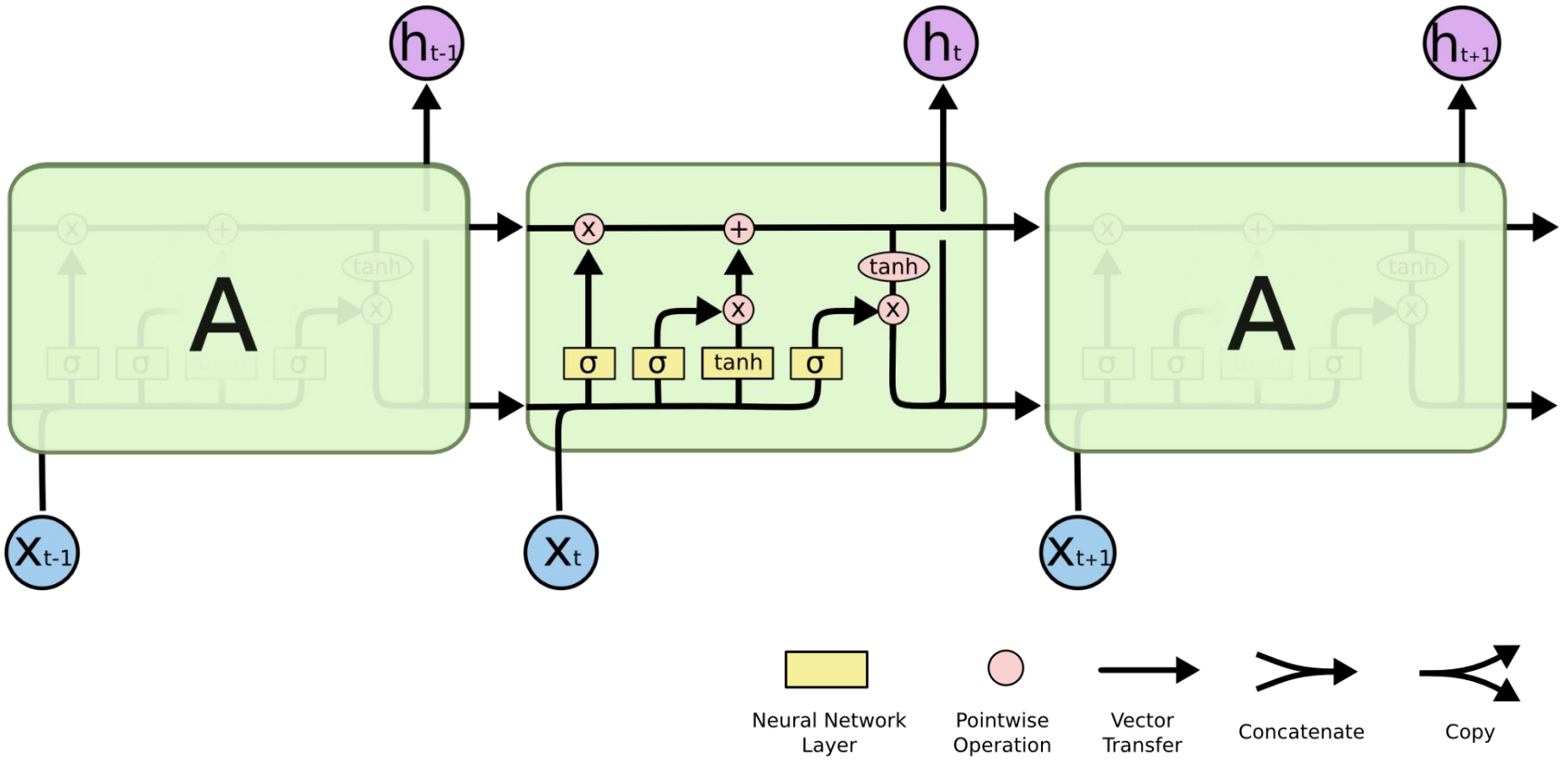
ReLU

$$\max(0, x)$$



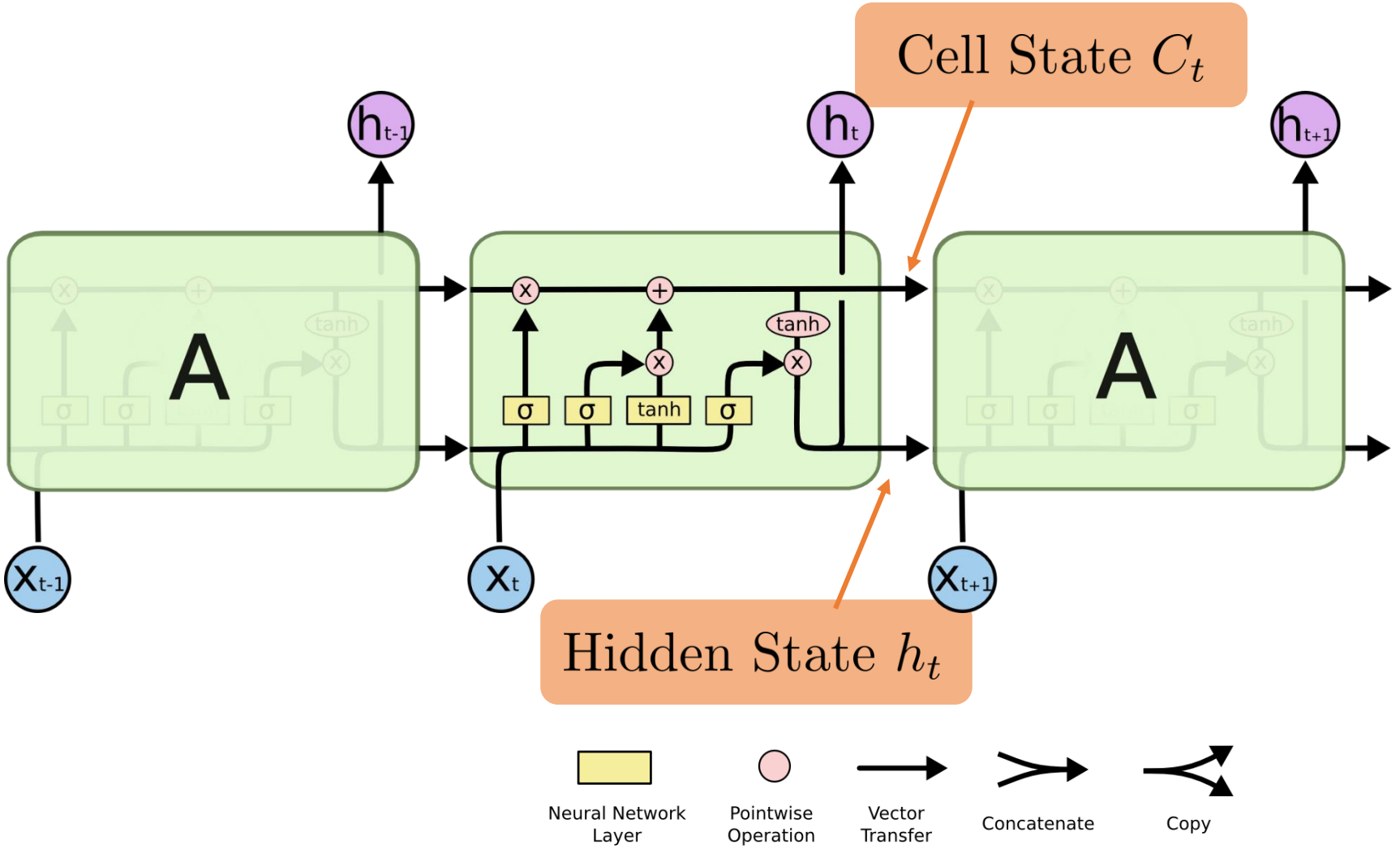
LSTM

- **典型的 LSTM**



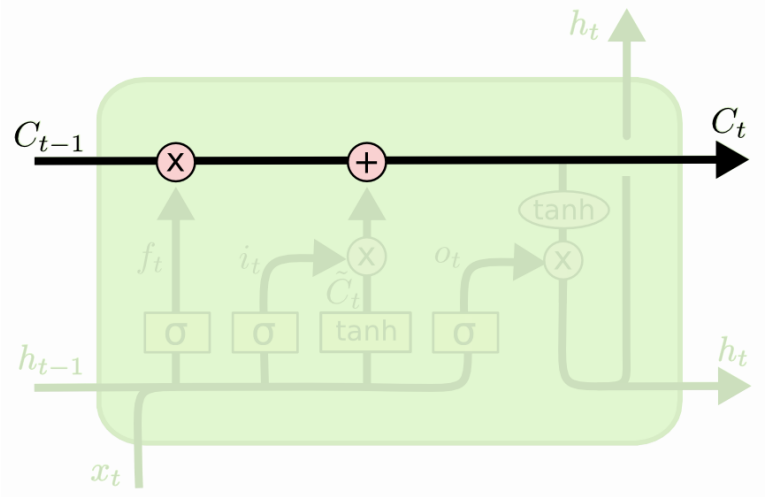
LSTM

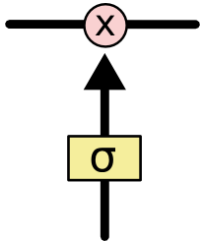
- **LSTM 的单元之间传递了两类状态**



LSTM 中的 Cell State

- **Cell State 是 LSTM 能实现其功能的关键所在**
  - 我们可以将其理解为一个“传送带”：它将关键的长期信息在整个序列中传递
  - 在传递中，每经过一个单元，信息都需要更新
    - 更新分为两步：筛选，和添加
    - 信息筛选是由一个被称为“门”（gate）的结构完成的





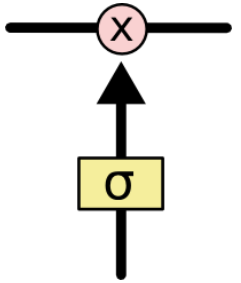
Gate

×表示向量中对应的元素相乘

σ是任意激活函数

LSTM 中的 Gates

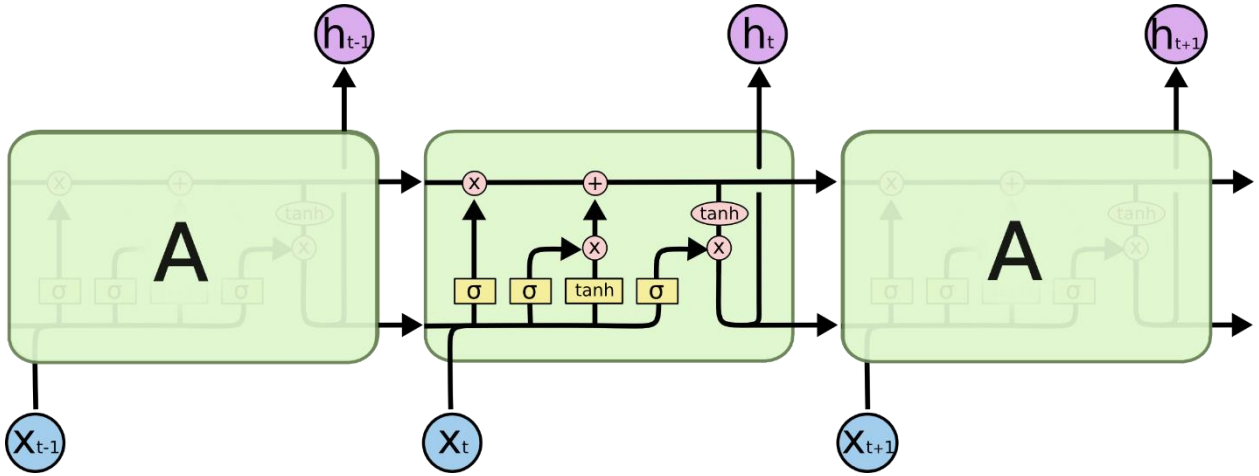
- **Gate 可以理解成对向量各元素施加相应的“权重”**
  - 权重接近1：通过
  - 权重接近0：不通过
- **LSTM 中使用了三个Gates**



Gate

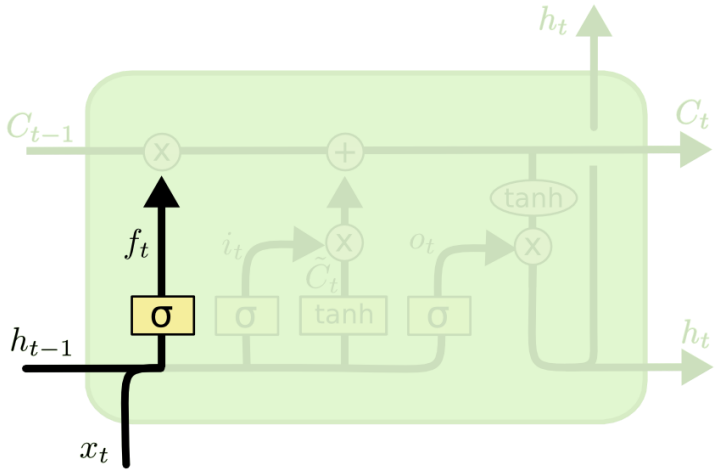
×表示向量中对应的元素相乘

σ是任意激活函数



LSTM 的各组成部分

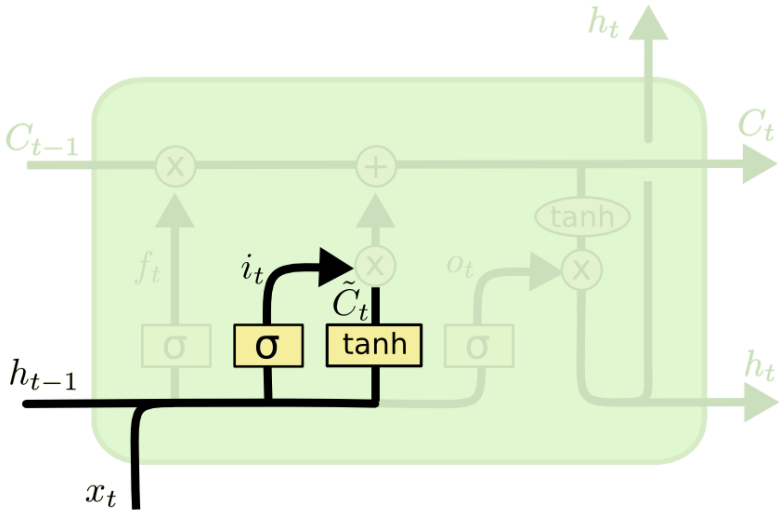
- **Forget Gate Layer：**
  - 生成“权重”：决定如何“筛选” Cell State 中的信息，即 Cell State 中哪些信息要保留，哪些要遗忘



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

LSTM 的各组成部分

- **Input Gate Layer：**
  - 处理输入信息和历史输出信息，生成可能要添加到 Cell State 的信息
  - 生成“权重”，决定哪些信息要添加到 Cell State

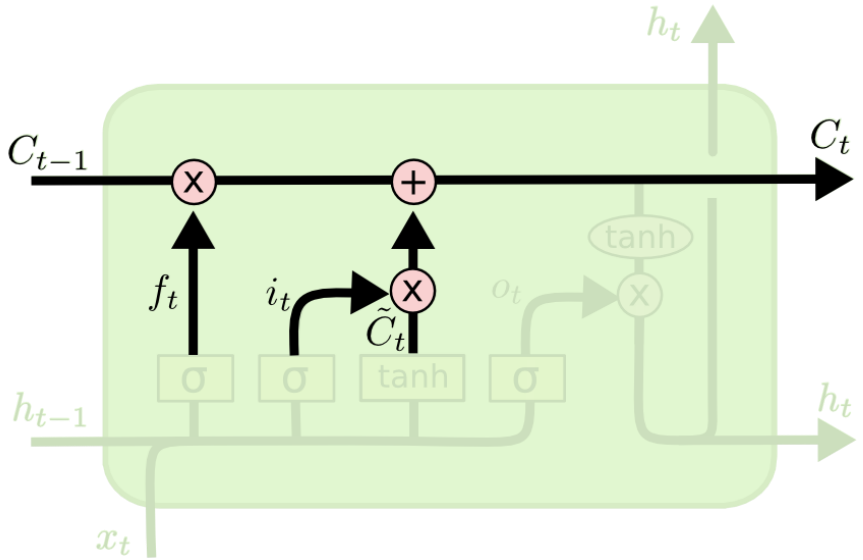


$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

LSTM 的各组成部分

- **Cell State Update**
  - 根据上两页的信息，对 Cell State 进行更新
    - 筛选，添加

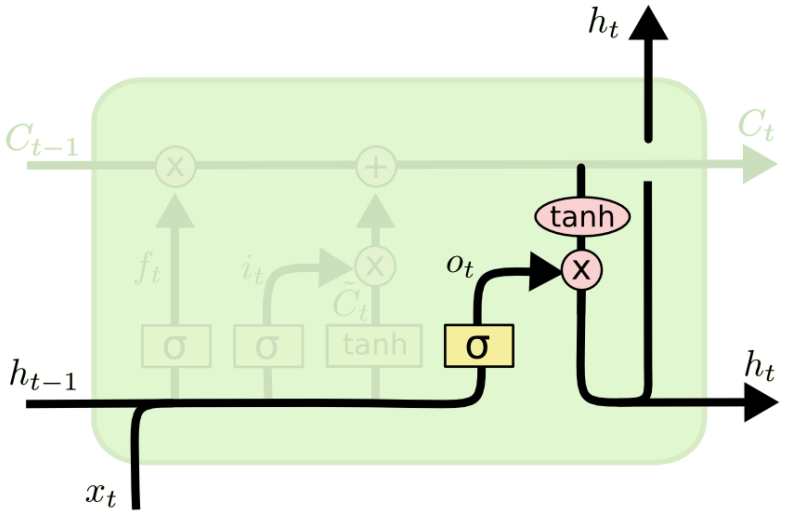


$$C_t = f_t \otimes C_{t-1} + i_t \otimes \tilde{C}_t$$

forget gate　previous cell states　input gate　new cell candidate

LSTM 的各组成部分

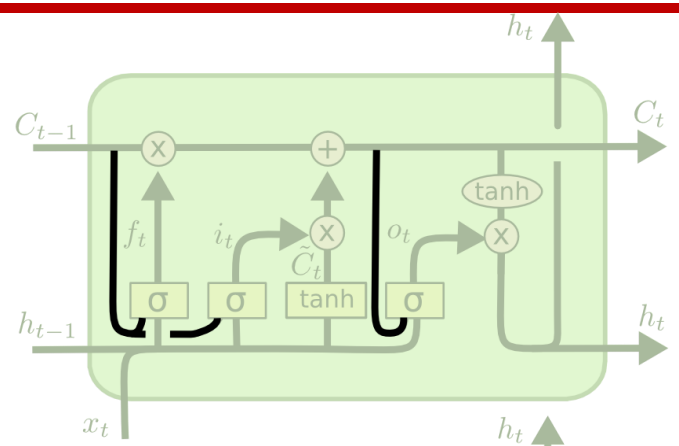
- **Output**
  - 输出的信息，是 Cell State 通过 tanh 激活函数，再经过一个 Gate
  - 该 Gate 由输入信息和历史输出信息决定



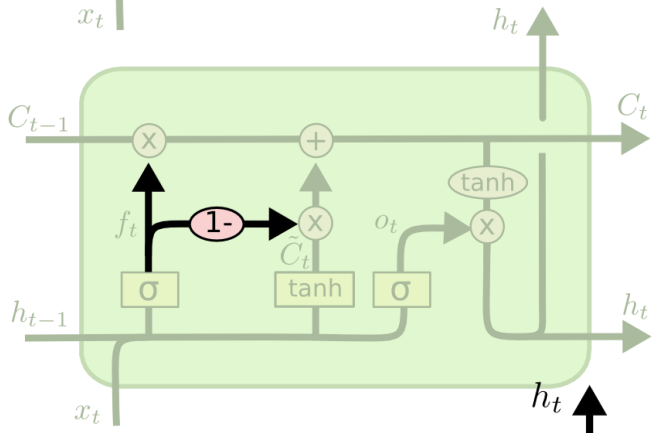
$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

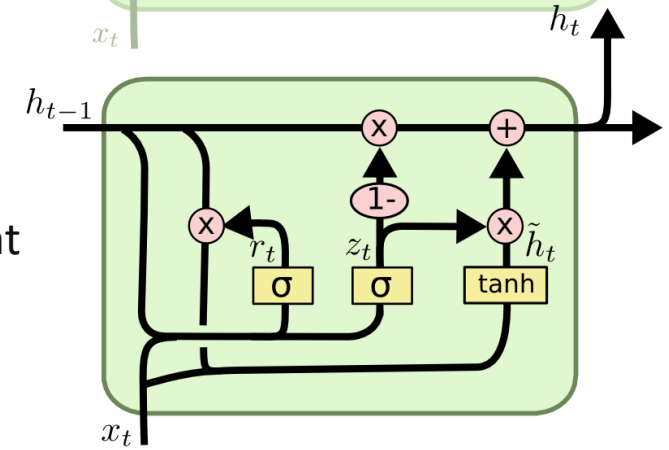
LSTM 的一些变体



$$f_t = \sigma\left(W_f \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \; + \; b_f\right)$$
$$i_t = \sigma\left(W_i \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \; + \; b_i\right)$$
$$o_t = \sigma\left(W_o \cdot [\boldsymbol{C_t}, h_{t-1}, x_t] \; + \; b_o\right)$$



$$C_t = f_t * C_{t-1} + (\mathbf{1} - \boldsymbol{f_t}) * \tilde{C}_t$$



Gate Recurrent Unit，GRU

$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$
$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$
$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$
$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

PyTorch 中使用 LSTM

LSTM

CLASS `torch.nn.LSTM(*args, **kwargs)` [SOURCE]

Applies a multi-layer long short-term memory (LSTM) RNN to an input sequence.

For each element in the input sequence, each layer computes the following function:

$$
\begin{aligned}
i_t &= \sigma(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi}) \\
f_t &= \sigma(W_{if}x_t + b_{if} + W_{hf}h_{t-1} + b_{hf}) \\
g_t &= \tanh(W_{ig}x_t + b_{ig} + W_{hg}h_{t-1} + b_{hg}) \\
o_t &= \sigma(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho}) \\
c_t &= f_t \odot c_{t-1} + i_t \odot g_t \\
h_t &= o_t \odot \tanh(c_t)
\end{aligned}
$$

where h_t is the hidden state at time *t*, c_t is the cell state at time *t*, x_t is the input at time *t*, h_{t-1} is the hidden state of the layer at time *t-1* or the initial hidden state at time *0*, and i_t, f_t, g_t, o_t are the input, forget, cell, and output gates, respectively. σ is the sigmoid function, and $\odot$ is the Hadamard product.

In a multilayer LSTM, the input $x_t^{(l)}$ of the l -th layer ($l >= 2$) is the hidden state $h_t^{(l-1)}$ of the previous layer multiplied by dropout $\delta_t^{(l-1)}$ where each $\delta_t^{(l-1)}$ is a Bernoulli random variable which is 0 with probability `dropout`.

If `proj_size > 0` is specified, LSTM with projections will be used. This changes the LSTM cell in the following way. First, the dimension of h_t will be changed from `hidden_size` to `proj_size` (dimensions of W_{hi} will be changed accordingly). Second, the output hidden state of each layer will be multiplied by a learnable projection matrix: $h_t = W_{hr}h_t$. Note that as a consequence of this, the output of LSTM network will be of different shape as well. See Inputs/Outputs sections below for exact dimensions of all variables. You can find more details in https://arxiv.org/abs/1402.1128.

PyTorch 中使用 LSTM

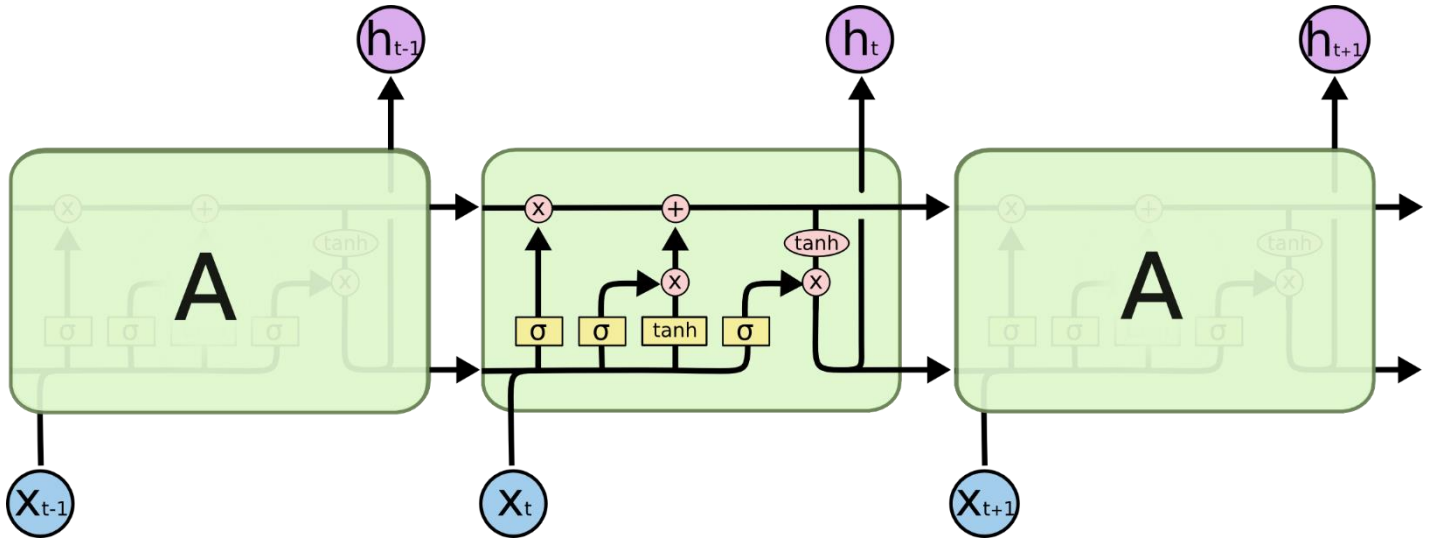
CLASS torch.nn.LSTM(*args, **kwargs)

Parameters

- **input_size** – The number of expected features in the input *x*
- **hidden_size** – The number of features in the hidden state *h*
- **num_layers** – Number of recurrent layers. E.g., setting num_layers=2 would mean stacking two LSTMs together to form a *stacked LSTM*, with the second LSTM taking in outputs of the first LSTM and computing the final results. Default: 1
- **bias** – If False, then the layer does not use bias weights *b_ih* and *b_hh*. Default: True
- **batch_first** – If True, then the input and output tensors are provided as *(batch, seq, feature)* instead of *(seq, batch, feature)*. Note that this does not apply to hidden or cell states. See the Inputs/Outputs sections below for details. Default: False
- **dropout** – If non-zero, introduces a *Dropout* layer on the outputs of each LSTM layer except the last layer, with dropout probability equal to dropout. Default: 0
- **bidirectional** – If True, becomes a bidirectional LSTM. Default: False
- **proj_size** – If > 0, will use LSTM with projections of corresponding size. Default: 0

LSTM 小结

- **LSTM 基于 RNN 结构，能更好地处理序列中需要长期记忆的信息**



Transformer **神经网络**

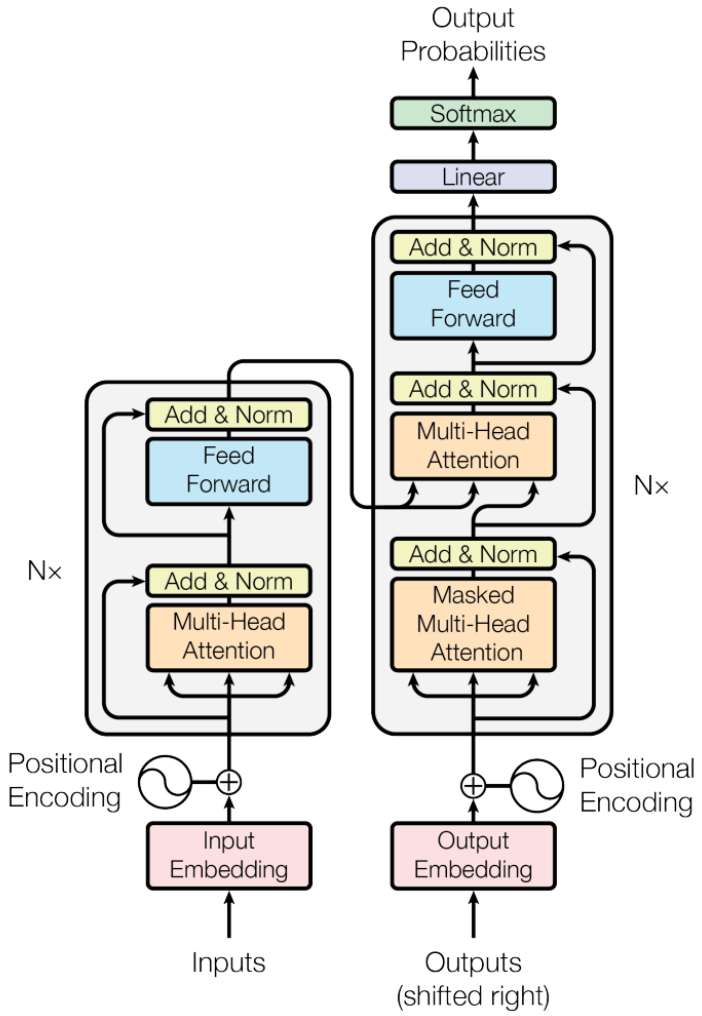
Transformer and Attention

- [1706.03762] Attention Is All You Need (arxiv.org)

- **Transformer 使用了注意力机制（attention mechanism）**

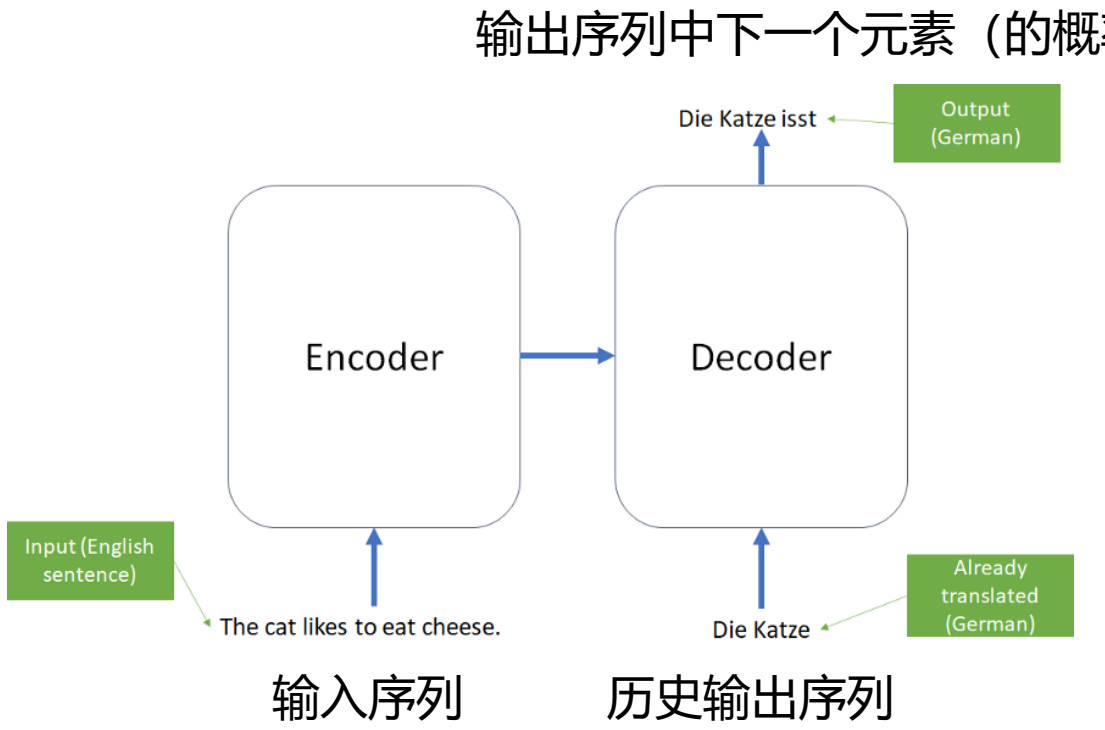
- **在大型的带有时序信息的模型中，Transformer 基本已取代了 LSTM 等基于 RNN 的方法**

Transformer 的基本结构

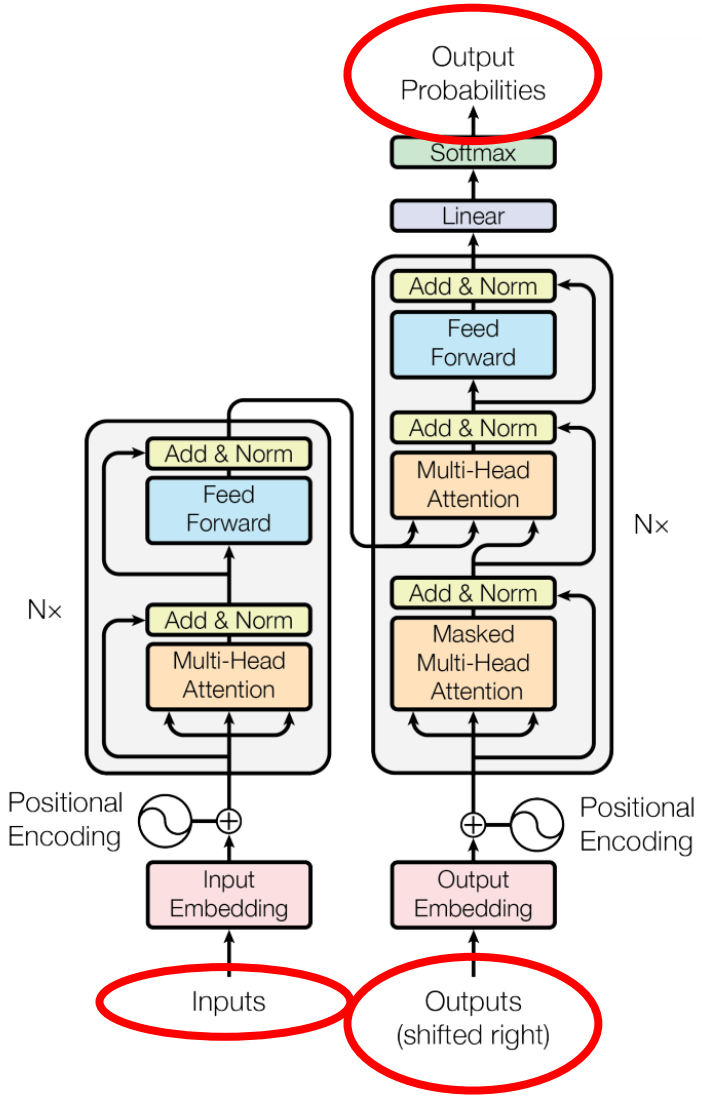


Transformer 的 Encoder – Decoder 结构

- **Transformer 的输入是整个序列（句子）**
- **Transformer 除了需要整个“输入序列”之外，还需要“历史输出序列”**

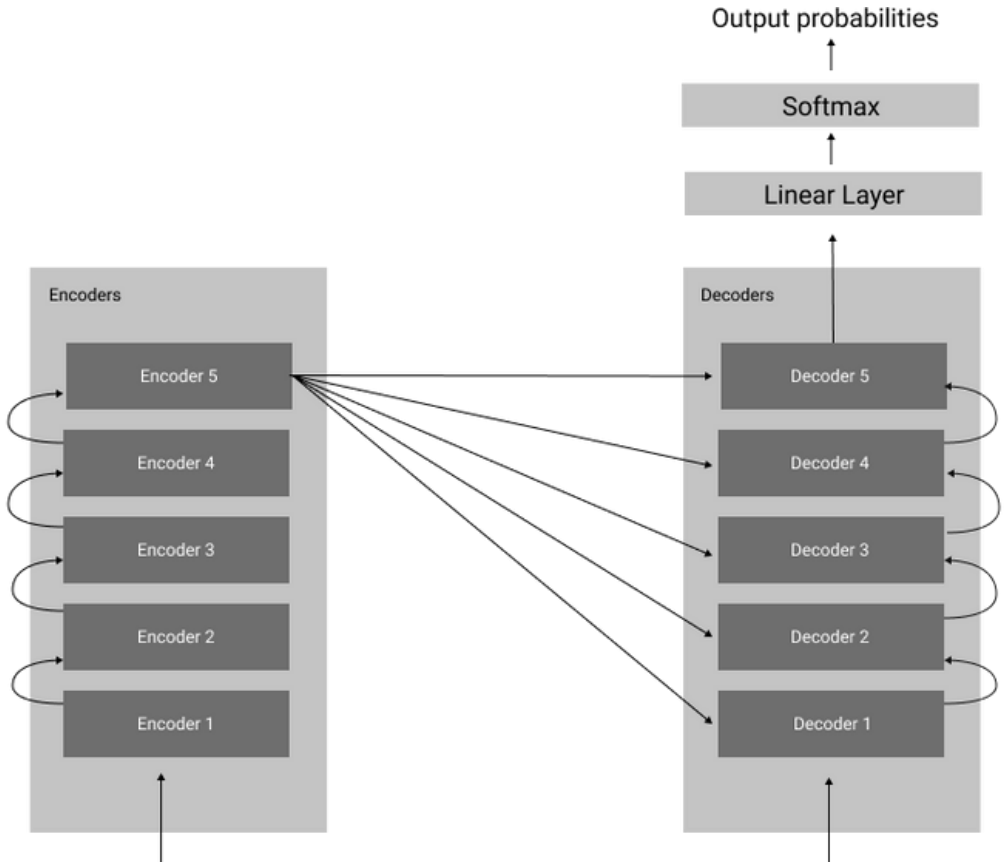


Transformer 的基本结构



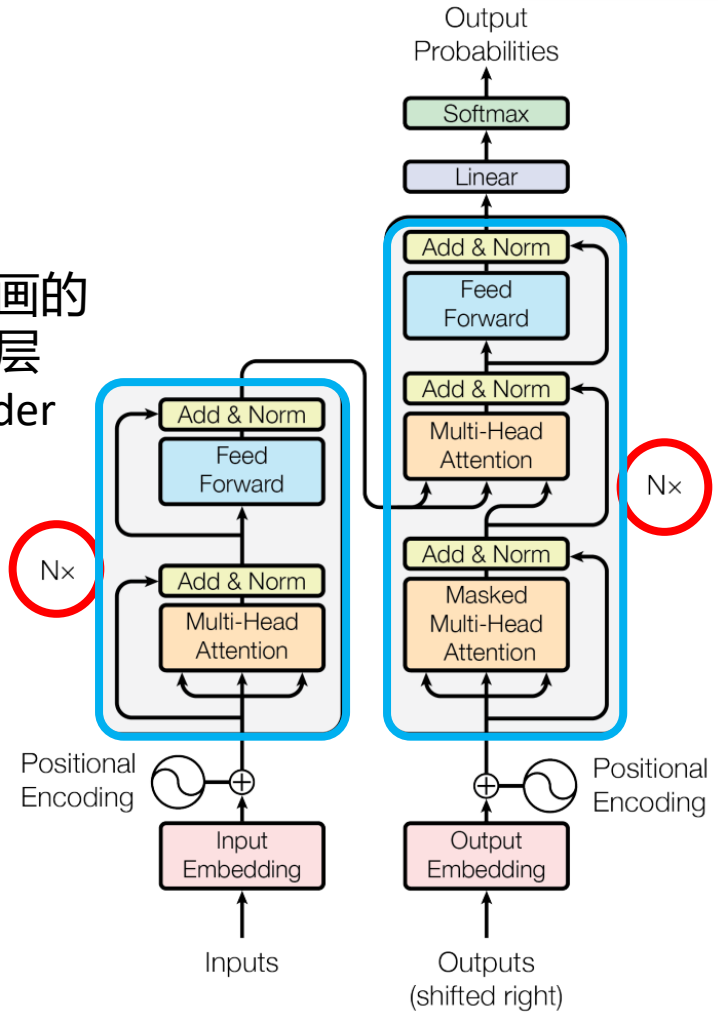
Transformer 的 Encoder – Decoder 结构

- **Encoder 和 Decoder 都有 N 层（下图中 N=5）**
  - 每层 Encoder 结构相同，参数不同
  - 每层 Decoder 结构相同，参数不同



Transformer 的基本结构

图中画的是一层 Encoder



图中画的是一层 Decoder

Transformer 中的注意力机制

- **注意力机制用到的名词，可类比以下这些概念**
  - Key：关键字
  - Value：取值；（Key，Value）通常成对出现
    - 例：（”张三”，99），（”李四”，100），（”王五”，99）
  - Query：查询
    - 例：给定上述三对（Key，Value），查询“张三”对应的数字

Transformer **中的注意力机制**

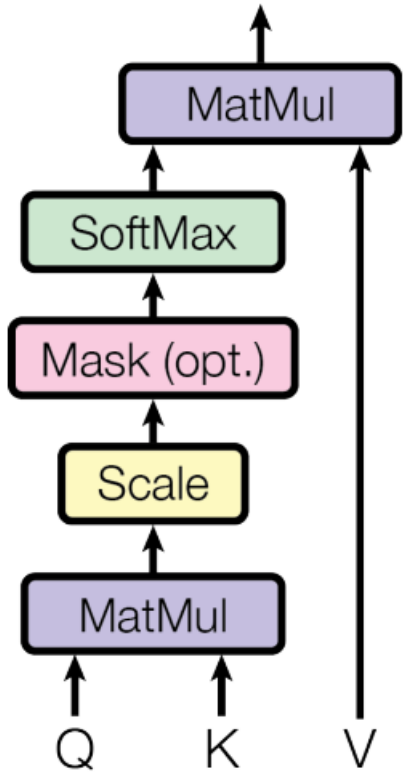
- **注意力函数**
  - 输入：一个 query，和若干个（key，value）组合
  - 输出：多个 values 的加权平均

- **在神经网络中，我们所说的 key，value 和 query 都是向量**

Transformer 中的注意力机制

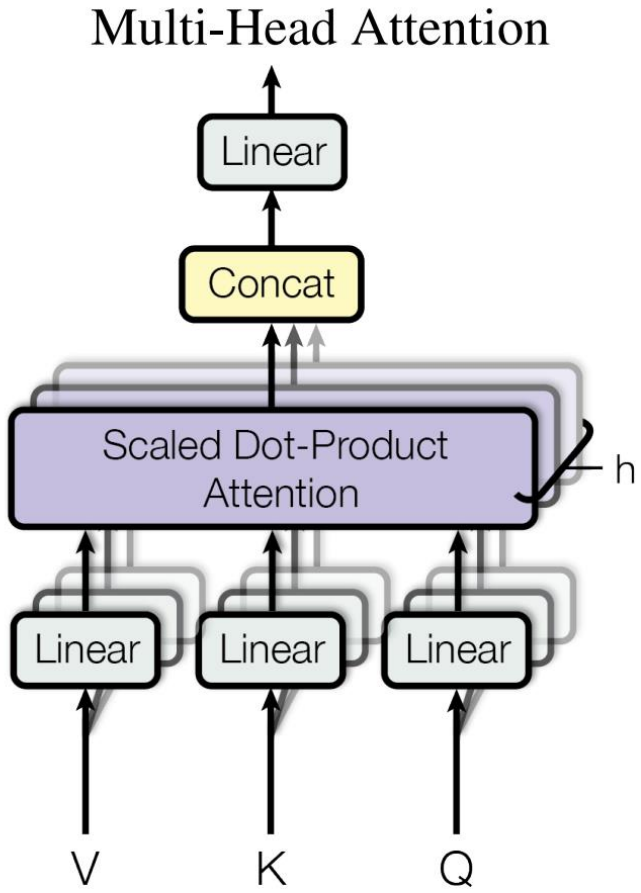
- **Scaled Dot-Product Attention**
  - 每个 query 和 key 的维度都是 d_k
  - 多行 queries 组成矩阵 Q
  - 多行 keys 组成矩阵 K
  - 多行 values 组成矩阵 V
  - 基于 Q 和 K 相乘来模拟“查询”

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

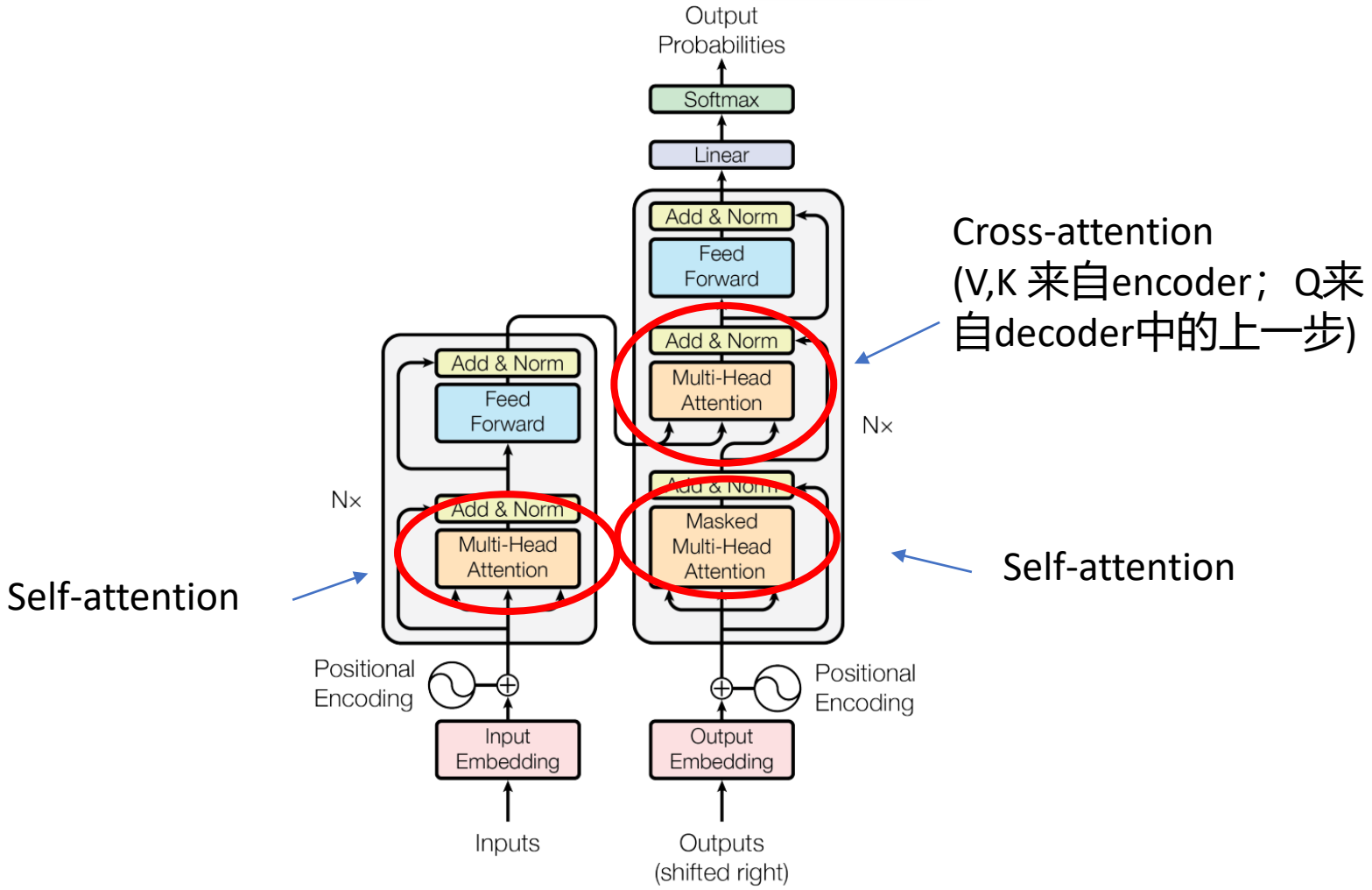


Transformer 中的注意力机制

- **Multi-Head Attention**
  - 对同一组高维的 V、K、Q，使用多个不同的线性函数，投影得到不同 d_v, d_k, d_k 维向量
  - Concat：向量拼接

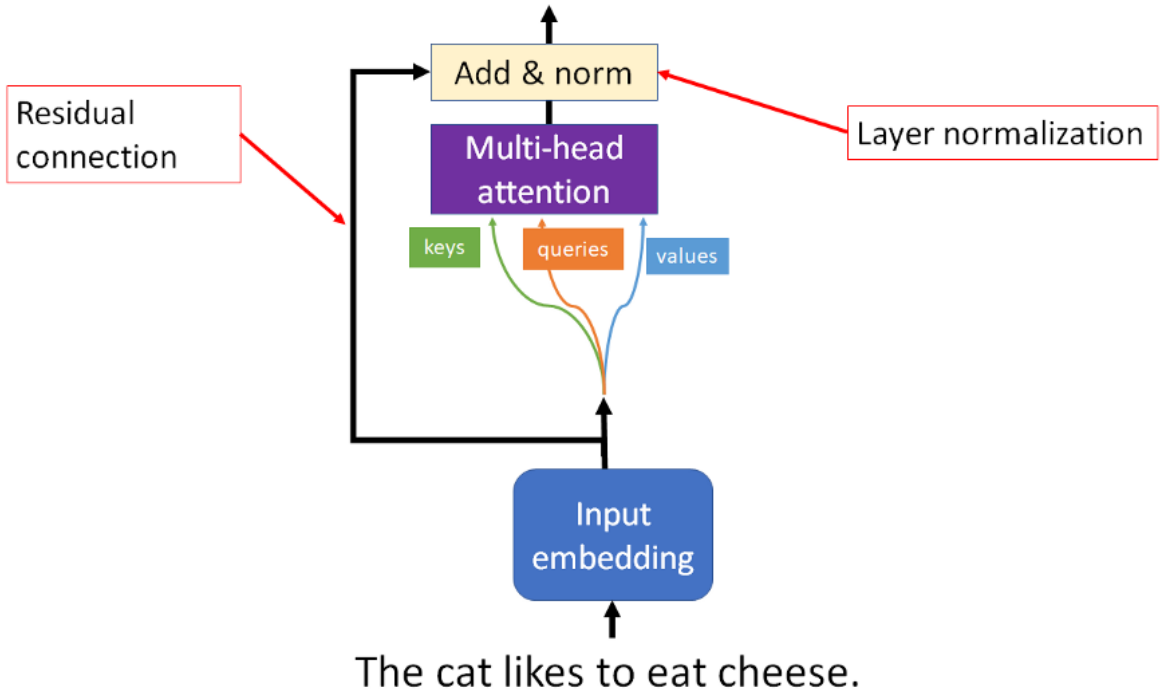


Transformer 的基本结构



Transformer

- **Add & norm：加入残差与进行归一化**

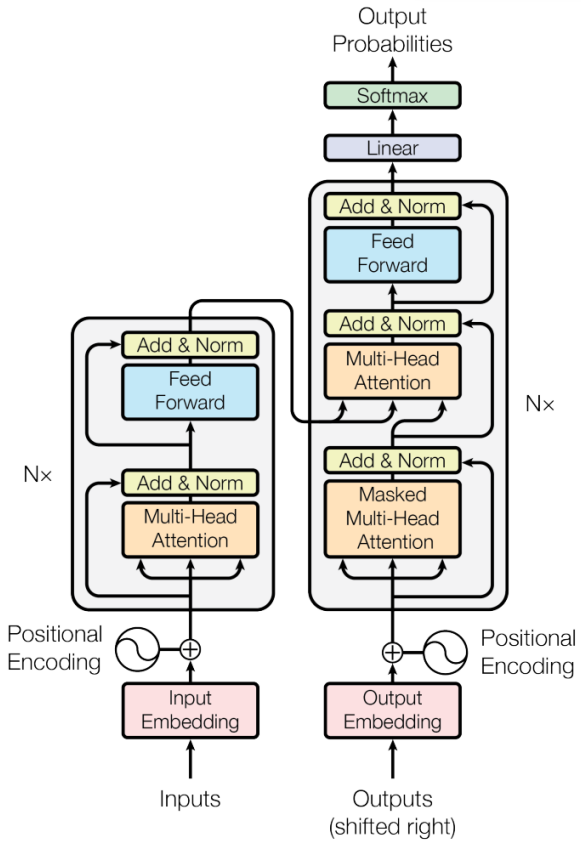


Transformer

- **Feed forward：两层线性变化，中间用 ReLu**

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- **Positional Encoding：给序列中的每个元素加入位置信息**
  - 注意力机制处理本身是不区分 inputs 顺序的
  - 原论文中使用的方法是加入不同频率的 sin、 cos 波形
  - 这里也可以使用其它的 Positional Encoding 方法



PyTorch 中使用 Transformer

TRANSFORMER

CLASS

torch.nn.Transformer(*d_model=512*, *nhead=8*, *num_encoder_layers=6*, *num_decoder_layers=6*, *dim_feedforward=2048*, *dropout=0.1*, *activation=<function relu>*, *custom_encoder=None*, *custom_decoder=None*, *layer_norm_eps=1e-05*, *batch_first=False*, *norm_first=False*, *device=None*, *dtype=None*)[SOURCE]

A transformer model. User is able to modify the attributes as needed. The architecture is based on the paper "Attention Is All You Need". Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In Advances in Neural Information Processing Systems, pages 6000-6010. Users can build the BERT(https://arxiv.org/abs/1810.04805) model with corresponding parameters.

Parameters

- **d_model** – the number of expected features in the encoder/decoder inputs (default=512).
- **nhead** – the number of heads in the multiheadattention models (default=8).
- **num_encoder_layers** – the number of sub-encoder-layers in the encoder (default=6).
- **num_decoder_layers** – the number of sub-decoder-layers in the decoder (default=6).
- **dim_feedforward** – the dimension of the feedforward network model (default=2048).
- **dropout** – the dropout value (default=0.1).
- **activation** – the activation function of encoder/decoder intermediate layer, can be a string ("relu" or "gelu") or a unary callable. Default: relu
- **custom_encoder** – custom encoder (default=None).
- **custom_decoder** – custom decoder (default=None).
- **layer_norm_eps** – the eps value in layer normalization components (default=1e-5).
- **batch_first** – If True, then the input and output tensors are provided as (batch, seq, feature). Default: False (seq, batch, feature).
- **norm_first** – if True, encoder and decoder layers will perform LayerNorms before other attention and feedforward operations, otherwise after. Default: False (after).

PyTorch 中使用 Transformer

Examples::

```
>>> transformer_model = nn.Transformer(nhead=16, num_encoder_layers=12)
>>> src = torch.rand((10, 32, 512))
>>> tgt = torch.rand((20, 32, 512))
>>> out = transformer_model(src, tgt)
```

LSTM 与 Transformer 在其它领域的应用

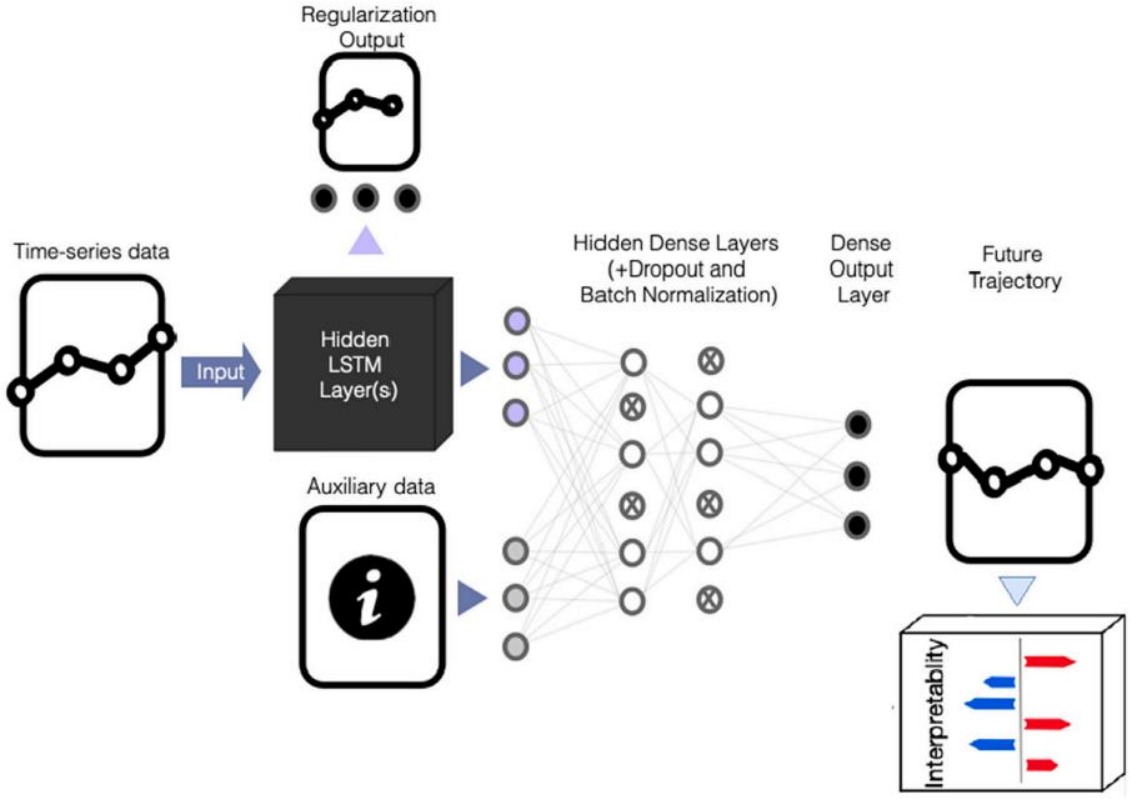
序列数据（sequential data）

- **LSTM 与 Transformer 在自然语言处理相关的应用中取得了良好的效果**
  - 它们都能很好地处理序列数据

- **动态系统中有大量与时间序列相关的问题**

例：轨迹预测

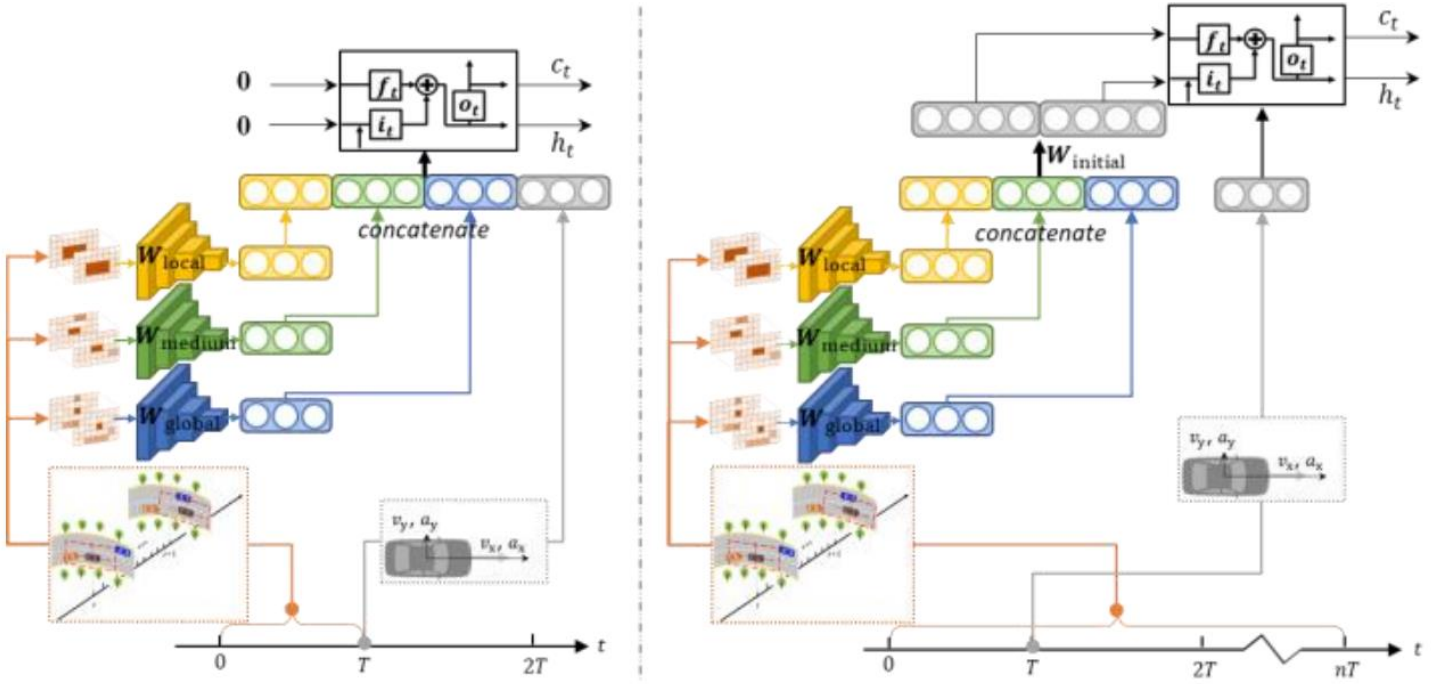
- LSTM 可用于复杂交通环境中车辆、行人的轨迹预测



A. Kalatian, “A context-aware pedestrian trajectory prediction framework for automated vehicles,” p. 21, 2022.

例：轨迹预测

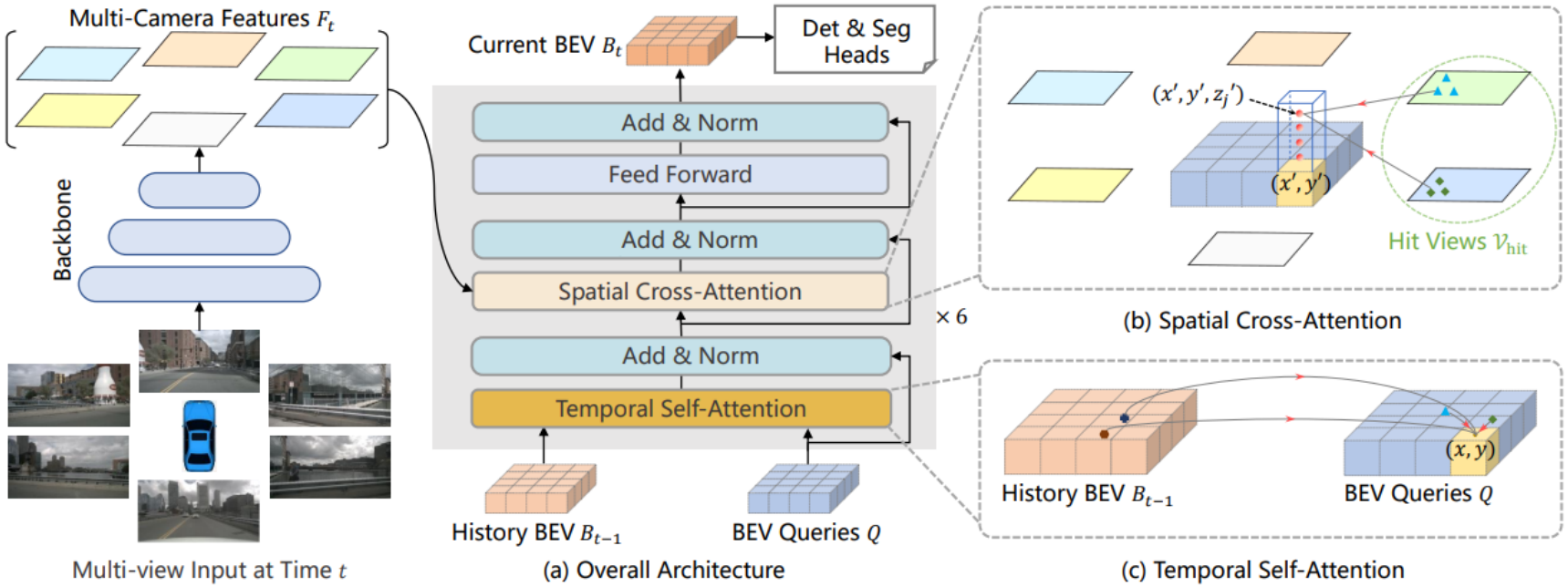
- LSTM 可用于复杂交通环境中车辆、行人的轨迹预测



L. Hou, S. E. Li, B. Yang, Z. Wang, and K. Nakano, “Integrated Graphical Representation of Highway Scenarios to Improve Trajectory Prediction of Surrounding Vehicles,” *IEEE Trans. Intell. Veh.*, pp. 1–14, 2022, doi: 10.1109/TIV.2022.3197179.

例：自动驾驶环境感知

- Transformer 可用于环境感知中对时间、空间序列数据（如多个摄像头的数据）的处理



Z. Li *et al.*, “BEVFormer: Learning Bird’s-Eye-View Representation from Multi-Camera Images via Spatiotemporal Transformers.” arXiv, Jul. 13, 2022. Accessed: Aug. 23, 2022. [Online]. Available: http://arxiv.org/abs/2203.17270

大语言模型
Large Language Models（LLMs）

大语言模型

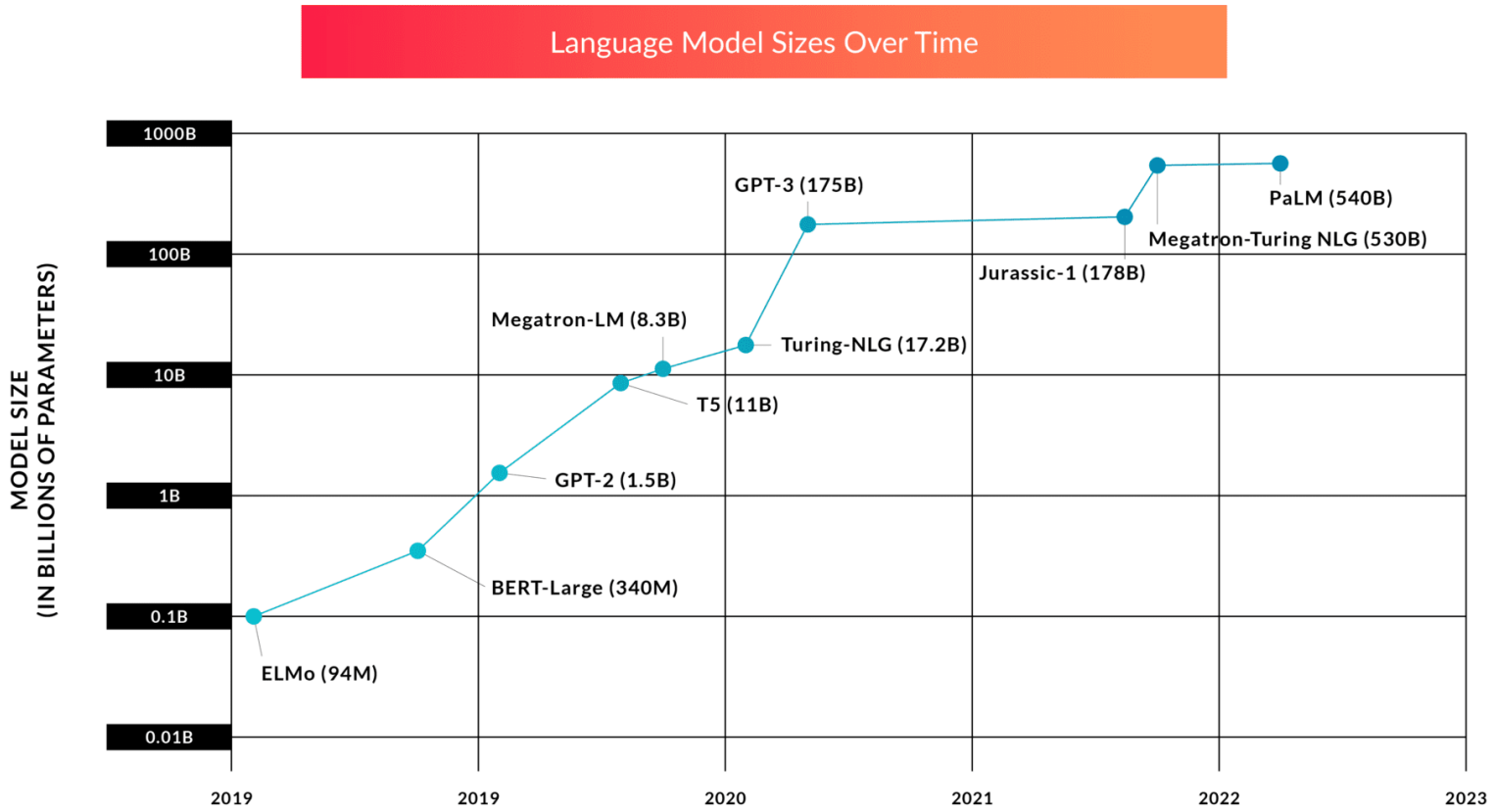
- 大语言模型由包含数百亿以上权重的深度神经网络构建，使用自监督学习为主的方式进行训练

- 大语言模型能理解、预测并生成自然语言文本



大模型参数量巨大

- 当前最先进的大模型参数量已接近 1000 Billion
  - 比人类大脑神经元的突触数量只少2~3个数量级



大语言模型中的涌现（emergent）现象

- 模型参数量增大到一定程度后，模型的效果突然大幅提升

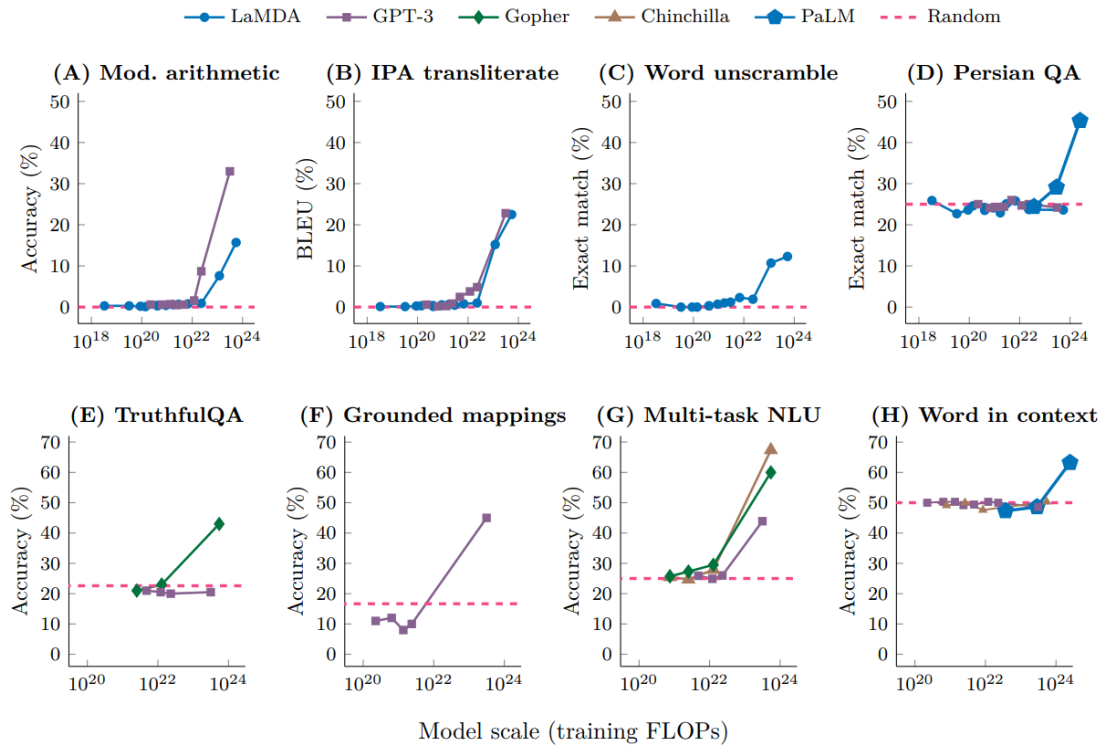


Figure 2: Eight examples of emergence in the few-shot prompting setting. Each point is a separate model. The ability to perform a task via few-shot prompting is emergent when a language model achieves random performance until a certain scale, after which performance significantly increases to well-above random. Note that models that used more training compute also typically have more parameters—hence, we show an analogous figure with number of model parameters instead of training FLOPs as the x-axis in Figure 11. A–D: BIG-Bench (2022), 2-shot. E: Lin et al. (2021) and Rae et al. (2021). F: Patel & Pavlick (2022). G: Hendrycks et al. (2021a), Rae et al. (2021), and Hoffmann et al. (2022). H: Brown et al. (2020), Hoffmann et al. (2022), and Chowdhery et al. (2022) on the WiC benchmark (Pilehvar & Camacho-Collados, 2019).

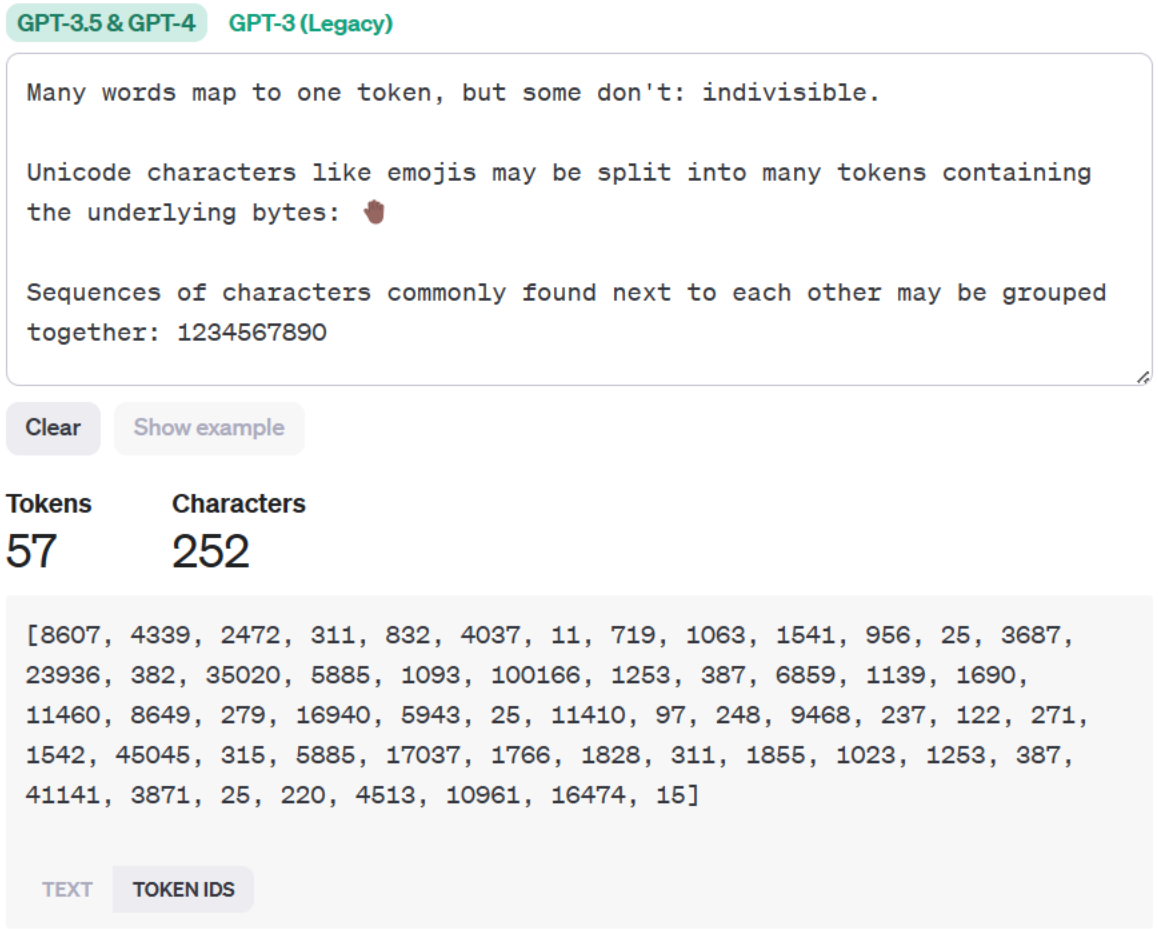
Tokenization：从文字到 Tokens

- **语言模型都需要用数字表达文字**
- **大语言模型的输入是由文字转化而成的 tokens，而非词嵌入向量**
  - 大模型内部的“词嵌入”部分（即找寻词与词之间联系的部分）不再需要单独训练；它们是模型架构的一部分

- **Token 是自然语言处理的最细粒度**
- **大语言模型的输入是 token 序列，输出也是 token 序列**

Tokenization：从文字到 Tokens

- OpenAI Tokenizer 显示 GPT-4 模型中从文字到 tokens 的例子



Tokenization：从文字到 Tokens

- **每个 token 是一个整数**
  - 这一步骤中不需要 word2vec 等方法来找词与词之间的联系

- **每个 token 对应一定的语义单元，而非限于单词**
  - 假如 token 只对应到单词，那么该模型就难以处理新的、未见过的词汇（如姓名等专有名词）
  - 通常 token 都是子词（sub-word）粒度
  - 标点符号等也有对应的 token

Tokenization：从文字到 tokens

- **一般来讲，每个英文单词至少对应1个 token**
  - 例：red → 1个 token
    - 在 GPT-3.5 与 GPT-4 中对应 [1171]
  - 例： statistical → 2个 tokens，”stat”+”istical”
    - 在 GPT-3.5 与 GPT-4中对应 [10079, 40596]

- **在 GPT-4 中，100 个 tokens ≈ 75 个英文单词**
  - 每个 token 大约对应 0.75 个英文单词

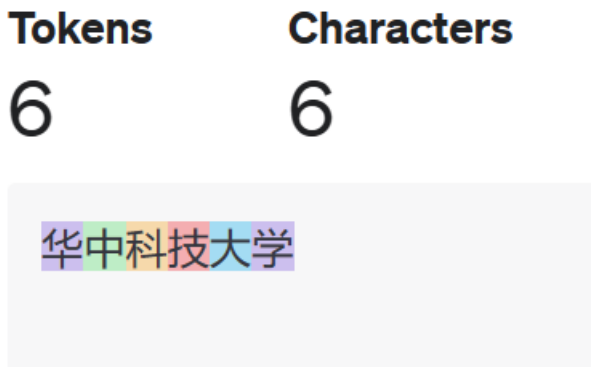
Tokenization：从文字到 tokens

- OpenAI Tokenizer 显示 GPT-4 模型中从文字到 tokens 的对应关系



Tokenization：从文字到 tokens

- 对中文而言，可将每个汉字对应为1个token



- 也有模型将中文里部分常用词作为1个token

Tokenization：**从文字到** tokens

- **还有一些大模型将一个汉字对应为多个 tokens**
  - 有些国外模型为了更好地支持多国文字，先对汉字使用 UTF-8 编码，将一个汉字编码为1到4个字节不等的序列
  - 此时再进行 tokenization，会出现部分汉字对应多个 tokens 的情况

大语言模型结构

- **大语言模型使用的神经网络，一般都使用了原始 Transformer 结构中的部分技术**
  - GPT (Generative Pre-trained Transformer)：采用了 Transformer 的解码器（decoder）堆栈
  - BERT (Bidirectional Encoder Representations from Transformers)：采用了 Transformer 的编码器（encoder）堆栈
  - 文心一言（ERNIE）： 使用了改进的 Transformer 架构，加入了额外的语言理解机制

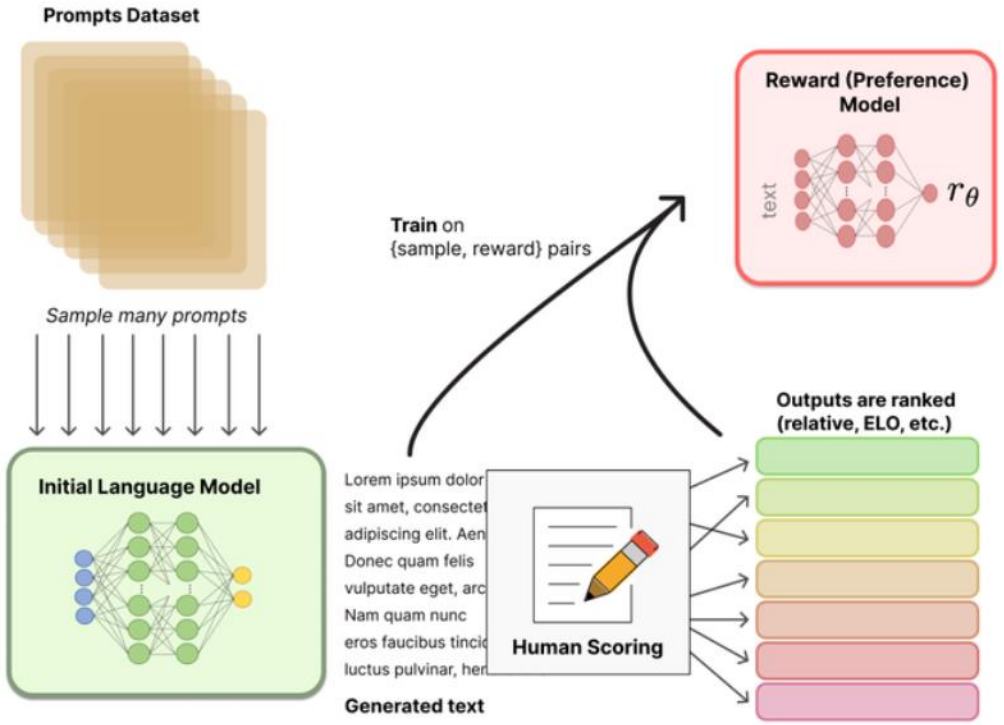
训练流程

- **第一步：预训练（pre-training）**
  - **自监督训练神经网络模型，预测一段 token 序列后的下一个 token**
    - 如 GPT-3 预测下一个 token 的概率，使用了交叉熵损失函数

- **第二步：微调（fine-tuning）**
  - **根据目标任务（如问答、情感分析、文本分类等），选择适当的标注数据集**
    - 数据集应该包含输入文本和期望输出的例子
  - **使用有监督的方式，进一步训练神经网络模型**
    - 微调时通常使用比预训练阶段更小的学习率

训练 ChatGPT 的额外步骤

- **（第三步）强化学习微调**
  - 通过与标注者的互动，采用奖励模型来进一步微调对话策略



分布式训练

- **大语言模型参数量太大，远超单块显卡的显存**
  - GPT-3 的 175 Billion 参数本身就要占用约 700GB 的显存，梯度下降中需要用到的显存还要再高数倍

- **大语言模型训练计算量太大，单机训练太慢**
  - 据估计，假如能用单块英伟达 A100 显卡训练 175B 的 GPT-3 模型，其耗时也需要 288 年

分布式训练

- 大语言模型都是在有多张显卡/通用计算单元的服务器上进行训练



Google TPU
(Tensor Processing Unit)



NVIDIA A100



服务器机柜

- 大模型推理通常也在服务器上进行